OpenDomain: A Domain-First, Evidence-Driven Framework for Engineering Software with AI Coding Assistants

DRAFT — NOT YET EMPIRICALLY VALIDATED

Version 1.1 · 2026-09-28

```
Version:      1.1 (reference implementation complete; see Section 12)
Format:       plain text
Date:         2026-09-28
Status:       DRAFT -- NOT YET EMPIRICALLY VALIDATED
Artifact role:this document is the authoritative specification of intent for the
              OpenDomain framework. Per OpenDomain's own epistemic discipline
              (Section 5.2), it is classified DRAFT / INFERRED and contains no
              validated empirical claims. Section 12 records precisely which
              claims are unvalidated.
```

Abstract

We present OpenDomain, a domain-first, evidence-driven software engineering framework for use with AI coding assistants ("vibe coding"). The framework addresses a specific and increasingly common failure mode: an AI assistant can produce syntactically complete, internally consistent, test-passing software whose domain semantics are invented rather than known. The failure is not a modelling error; it is an epistemic error. The system has no representation of what is known, what is assumed, what is inferred, and what is unknown, and therefore no way to distinguish a reconstructed domain rule from a plausible hallucination. OpenDomain contributes (C1) a closed epistemic lattice attached to every engineering artifact, permitting EXPLICIT, TACIT, HYBRID, INFERRED, ASSUMED, UNKNOWN and CONFLICTING knowledge to be represented, preserved and enforced as distinct states rather than collapsed into a single asserted specification; (C2) an evidence-gated derivation protocol that structurally forbids promoting an external statement directly into a requirement, requiring the intermediate steps Source → Finding → Interpretation → Candidate Need → Requirement; (C3) a single ordered traceability spine spanning source, evidence, domain knowledge, requirement, use case, specification, acceptance criterion, contract, plan, task, code, test and validation result, over which two query classes – "Why does this exist?" and "If this changes, what breaks?" – become mechanically answerable; (C4) nine deterministic, side-effect-free quality gates that detect epistemic laundering, evidence-free requirements and dangling traceability links before implementation begins; (C5) a first-class opportunity stage that formalises innovation as a falsifiable, kill-criteria-governed engineering process, preventing feature replication from masquerading as analysis; and (C6) an explicit formal rejection of the Minimum Viable Product as a scope-governing device, replaced by evidence-bounded capability classification and evidence-complete vertical increments. Two contributions concern verification specifically. (C7) a three-loop process that nests specification-driven, behavioural-driven and test-driven development as concentric loops with different units of verification, different artifacts and different definitions of done, together with a rigour-by-requirement-type table that makes the choice of rigour an explicit, reviewable declaration rather than an unstated habit. (C8) a provisioning subsystem that detects the project's language stack, selects a verification toolchain by a deterministic rule, installs it locally and idempotently at pinned versions, generates the executable specifications and step definitions, binds each verification artifact to the acceptance criterion it checks with recorded binding provenance, and refuses to proceed when the declared rigour cannot be honoured – while recording, for every specification, whether it was actually executed, merely discovered, or only declared. We formalise the framework as a nine-gate pipeline with explicit transition preconditions, specify a reference implementation architecture in which all gate evaluation is a pure function of a version-controlled workspace, and define an evaluation protocol comprising four studies: a longitudinal case study, a controlled comparison against specification-first development, a falsification-oriented survey of constructed domain artefacts, and a mechanised audit of epistemic-state transitions. We report the framework

design, the gate semantics, the artifact model, and the protocol. We do not report results; Section 12 states what remains unvalidated.

Keywords: domain engineering, requirements engineering, evidence-driven

Contents

1 INTRODUCTION **6**

1.1 The setting 6
1.2 The specific failure mode 7
1.3 Why existing process knowledge does not solve it 7
1.4 What this paper contributes 8
1.5 Scope 8

2 PROBLEM STATEMENT **8**

2.1 Formal statement 8
2.2 The under-specified agent 9
2.3 Two concrete failure instances 9

3 RESEARCH QUESTIONS AND DESIGN OBJECTIVES **9**

3.1 The two motivating goals 9
3.2 Research questions 10
3.3 Non-goals 10

4 BACKGROUND AND RELATED WORK **11**

4.1 Domain knowledge in requirements engineering 11
4.2 Specification-driven development 11
4.3 Domain-specific modelling languages and MBSE 11
4.4 Domain-driven design 12
4.5 Specification by example, BDD and TDD 12
4.6 The Minimum Viable Product and its limits 13
4.7 Position summary 13
4.8 Mapping from founding operational protocols 13

5 THE OPENDOMAIN FRAMEWORK **13**

5.1 The pipeline 13
5.2 The epistemic lattice 14
5.3 The evidence gate on derivation 15
5.4 The traceability spine 15
5.5 Bidirectionality and the pre-requirements question 16
5.6 Cross-domain analysis and concept ownership 16
5.7 The opportunity stage 17

5.8 Uncertainty as a managed, not suppressed, quantity . . . 17
5.9 Architecture is derived, not selected . . . 18
5.10 Brownfield: existing code is evidence, not specification . . . 18
5.11 The minimal-artifact principle . . . 18
5.12 The framework's own audit . . . 18

6 ARTIFACT MODEL AND IDENTIFIER SYSTEM 19
6.1 Identifier kinds . . . 19
6.2 Artifact statuses . . . 19
6.3 Artifact record structure . . . 19
6.4 Authority and confidence . . . 20

7 THE NINE QUALITY GATES 20
7.1 Design constraint . . . 20
7.2 Gate 1 – context . . . 20
7.3 Gate 2 – research . . . 21
7.4 Gate 3 – domain . . . 21
7.5 Gate 4 – requirements . . . 22
7.6 Gate 5 – opportunity . . . 22
7.7 Gate 6 – specification . . . 23
7.8 Gate 7 – plan . . . 23
7.9 Gate 8 – implementation . . . 24
7.10 Gate 9 – completion . . . 25
7.11 Gate summary . . . 25

8 REJECTING THE MINIMUM VIABLE PRODUCT 26
8.1 Position . . . 26
8.2 What MVP does well . . . 26
8.3 Why its preconditions fail here . . . 26
8.4 What replaces it . . . 26
8.4.1 Evidence-bounded capability classification . . . 27
8.4.2 The evidence-complete increment . . . 27
8.4.3 MVP retained where it is actually valid: the experiment . . . 27
8.5 Consequences accepted . . . 28
8.6 What would falsify this position . . . 28

9 THE TDD / BDD / SDD PROCESS 28
9.1 The error this section corrects . . . 28
9.2 The three loops, defined . . . 29
9.3 The outer loop: SDD . . . 29
9.4 The middle loop: BDD . . . 29
9.5 The inner loop: TDD . . . 30
9.6 Handoff contracts . . . 30
9.7 Rigour selection . . . 31
9.8 Three misuses and their detectable signatures . . . 31
9.9 The contested-rule row . . . 31
9.10 Executable specifications . . . 32

10 AUTOMATIC VERIFICATION TOOLCHAIN PROVISIONING 32
10.1 The provisioning problem . . . 32
10.2 Six provisioning principles . . . 32
10.3 P0 – stack detection . . . 33
10.4 P1 – deterministic framework selection . . . 33
10.5 The observation that makes this cheap . . . 34
10.6 P2 – installation . . . 34
10.7 P3 – wiring generation . . . 34
10.8 P4 – traceability binding, and what it buys . . . 35
10.9 Verification depth – the necessary caveat . . . 36
10.10 P5 – unit-level binding, and binding provenance . . . 36
10.11 P6 – contract testing . . . 36
10.12 Brownfield: extract, do not install . . . 37
10.13 The provisioning gate . . . 37
10.14 Limits and failure modes . . . 37
10.15 The adapted lifecycle, end to end . . . 38

11 REFERENCE IMPLEMENTATION ARCHITECTURE 39
11.1 Purpose and stance . . . 39
11.2 Component decomposition . . . 40
11.3 The purity constraint . . . 40
11.4 Determinism . . . 40
11.5 Extension points and their limits . . . 40
11.6 Implementation status . . . 41
11.7 Traceability of this paper to the implementation . . . 41

12 EPISTEMIC STATUS OF THIS PAPER 42

13 EVALUATION PROTOCOL **42**
13.1 S1 – Longitudinal case study (addresses RQ1, RQ2, RQ6) 43
13.2 S2 – Controlled comparison against MVP scoping (addresses RQ5) 43
13.3 S3 – Epistemic-state audit (addresses RQ1, RQ3) 44
13.4 S4 – Gate utility (addresses RQ3, RQ6) 44
13.5 Instrumentation 44

14 THREATS TO VALIDITY **44**
14.1 Threats to construct validity 44
14.2 Threats to internal validity 44
14.3 Threats to external validity 45
14.4 Threats to conclusion validity 45

15 DISCUSSION **45**
15.1 What the framework changes about the agent's role 45
15.2 The framework as a research instrument 45
15.3 The cost objection, taken seriously 45
15.4 On rejecting MVP 46
15.5 The three-stage separation 46

16 LIMITATIONS **46**

17 ETHICS AND SUSTAINABILITY **47**
17.1 Epistemic honesty as an engineering duty 47
17.2 The risk of the framework being misused 47
17.3 Sustainability of the knowledge substrate 48

18 FUTURE WORK **48**

19 CONCLUSION **49**

References **49**

A Appendix A – ARTIFACT FIELD SCHEMAS **50**
A.1 Common fields (all kinds) 50
A.2 Kind-specific requirements 51
A.3 What is deliberately absent 51

B Appendix B – GATE PRECONDITION TABLES **51**
B.1 Preconditions 51
B.2 A note on gate ordering 52

C Appendix C – WORKED MICRO-EXAMPLE 52

C.1 The scenario 52

C.2 Gate 2 (research) – what the sources say 52

C.3 Gate 4 (requirements) – the evidence gate in action 53

C.4 Gate 6 (specification) – acceptance criteria 53

C.5 Gate 8 (implementation) – the laundering check 54

D Appendix D – MAPPING TO THE FOUNDING OPERATIONAL PROTOCOLS 54

D.1 Domain-engineering protocol (two variants) 54

D.2 Spec-driven-engineering protocol 54

D.3 Opportunity / innovation-engineering protocol 55

D.4 Automatic verification toolchain provisioning 56

D.5 Net effect of the mapping 56

1. Introduction
2. Problem statement
3. Research questions and design objectives
4. Background and related work
5. The OpenDomain framework
6. Artifact model and identifier system
7. The nine quality gates
8. Rejecting the Minimum Viable Product
9. The TDD / BDD / SDD process
10. Automatic verification toolchain provisioning
11. Reference implementation architecture
12. Epistemic status of this paper
13. Evaluation protocol
14. Threats to validity
15. Discussion
16. Limitations
17. Ethics and sustainability
18. Future work
19. Conclusion References Appendix A – Artifact field schemas Appendix B – Gate precondition tables Appendix C – Worked micro-example Appendix D – Mapping to the founding operational protocols and to the provisioning subsystem

1 INTRODUCTION

1.1 The setting

AI coding assistants have changed the entry cost of software production. A practitioner with limited or no software engineering training can now produce a running application from a natural-language description. This is a genuine democratisation of construction.

It has not democratised understanding.

The distinction matters because software engineering is not principally the activity of producing code. It is the activity of producing a defensible answer to the question "what should exist, and why?". A codebase that does not correspond to any defensible answer to that question is not

software; it is an artefact that behaves like software. The cost of such an artefact is deferred, not avoided: it surfaces during integration with the real world, during regulatory review, during staff turnover, and during the first requirement change that nobody anticipated.

1.2 The specific failure mode

We name the failure mode this paper addresses: semantic fabrication under plausible presentation.

An AI assistant asked to "add invoice management" will produce entities (Invoice, LineItem, Customer), a state machine, a REST API, migrations, and tests. These artifacts are internally coherent. They will also be, in the absence of explicit constraint:

- a reimplementation of the most common public invoice schema, not the schema of the deploying organisation;
- a lifecycle with four states, because four is the modal answer, not because the deploying organisation's fiscal and legal constraints imply four;
- tax handling that is plausible and non-compliant in the target jurisdiction;
- an approval workflow copied from competitor documentation, not derived from the actors' actual decision rights.

None of this is hallucination in the strict sense. Each individual statement is statistically reasonable. The defect is that the assistant has no representational slot in which to record that it does not know. It cannot distinguish:

```
"In Senegal, a facture normalisee must carry a NINEA and a registration
 number, per the Code general des impots" (known, and checkable)
```

from

```
"Invoices probably need approval states" (inferred from training
 distribution, and not checkable).
```

Both are emitted in the same register. The consumer of the output has no way to tell them apart. This is the core problem.

1.3 Why existing process knowledge does not solve it

The domain engineering, requirements engineering and specification-driven development literatures already contain most of the conceptual apparatus needed to solve this. What they do not contain is a mechanism that is simultaneously:

```
(a) machine-enforceable, so that an autonomous agent cannot bypass it by
    omission;
(b) cheap enough to be applied per-change rather than per-project;
(c) explicit about ignorance, so that uncertainty is a representable and
    reportable first-class state;
(d) agnostic to whether the specification is human-authored or
    agent-authored.
```

Process documents fail (a). Static analysis fails (c). Prompt engineering fails (b) and (d). Traceability matrices fail (b) unless automated. We argue that the missing element is not a new process stage but a new representational substrate: a machine-checkable artifact model in which epistemic state is part of the data rather than a matter of author discipline.

1.4 What this paper contributes

Contribution summary, stated conservatively. Each item is a design contribution; none is an empirical result.

```
C1. The epistemic lattice. A closed, seven-valued classification attached to
    every artifact, with no "default" value, such that an artifact cannot
    exist without declaring how it is known.

C2. The evidence gate on derivation. A structural constraint forbidding the
    promotion of an external statement to a requirement without the
    intermediate record of finding, interpretation and candidate need.

C3. The traceability spine. A single ordered artifact chain with a fixed
    relation vocabulary, supporting bidirectional query.

C4. Nine deterministic quality gates. Pure functions over a workspace, with
    no side effects, making gate results reproducible and auditable.

C5. The opportunity stage as a mandatory, hypothesis-governed stage,
    separating "what is true of the world" from "what we believe could be
    better", and making the latter falsifiable.

C6. The formal rejection of MVP-as-scope-device, and its replacement.

C7. Rigour selection as a function of requirement type rather than a global
    methodological mandate.
```

1.5 Scope

OpenDomain is a method and a reference implementation of that method. It is not a code generator, not a requirements management suite, and not a replacement for domain expert judgement. Where expert judgement is available, OpenDomain records and structures it; it does not manufacture it. Where it is unavailable, OpenDomain reconstructs from evidence and marks the residue as UNKNOWN. The framework's central claim is not that it eliminates ignorance but that it makes ignorance legible.

2 PROBLEM STATEMENT

2.1 Formal statement

Let D be a real-world domain. Let K be the knowledge an engineering process holds about D. Standard practice produces software S as a function of K without requiring K to be complete, explicit, or internally consistent:

```
S = f(K)      where K may be partial, tacit, contradictory, or absent
```

Under this regime the following are undecidable at any point in the lifecycle:

```
(Q1) Provenance.  For any element s in S, what observation or authority
                  justifies s?
(Q2) Necessity.   For any element s in S, is s required by the domain, or
                  has it been introduced by convention, analogy to a
                  competitor, or by an agent's prior?
(Q3) Blast radius. For any change c to a requirement r, which downstream
                  artifacts become invalid?
(Q4) Trust.       For any element s in S, is s confirmed, inferred, assumed,
                  or unknown?
```

We claim that Q1-Q4 are undecidable not because the information is unavailable but because the process has no representation in which to hold it. Information that is not represented is not recoverable.

2.2 The under-specified agent

The agent setting sharpens the problem. A human engineer applying domain engineering methods has, at minimum, tacit access to the organisation around them: they overhear a conversation, they see a form nobody uses, they know which manager does not read email. An AI assistant has no such access. Its entire epistemic endowment is its training distribution plus whatever the user supplies.

This produces a characteristic and dangerous asymmetry: the agent is *maximally fluent* precisely where it is *maximally ignorant*. Fluency is generated by the model; ignorance is not visible in the output because the output is fluent. Where a junior human engineer's ignorance is diagnosable – they hesitate, they ask, they get the terminology wrong – the agent's ignorance is syntactically indistinguishable from its competence.

Any framework intended for agentic use must therefore make ignorance *representable and checkable*, not merely discouraged. This is the design constraint from which most of OpenDomain follows.

2.3 Two concrete failure instances

Failure instance 1 – the tax rate. An assistant implements sales tax. The target jurisdiction uses value-added tax with a different rate structure, a different invoicing obligation, and possibly a different fiscal year boundary. The implementation is clean, tested, and wrong. The requirement "the system SHALL compute tax" has no jurisdiction, and the system has no notion that tax is jurisdiction-scoped. Nothing in the codebase records that this is unresolved.

Failure instance 2 – the approval step. An assistant adds status: DRAFT → PENDING_APPROVAL → APPROVED, copied from competitor documentation. In the target organisation, approval authority belongs to a role that the assistant has never heard of, and the approval is legally required to be performed by a named individual with a signature obligation. The state machine is a plausible fiction. The requirement derived from it will be "traceable" to a source that is, on inspection, a competitor's marketing page.

In both cases the conventional remedy – "write better requirements" – is ineffective, because the requirement was never wrong in the sense of being ambiguous. It was wrong in the sense of being unfounded, and nothing in its form revealed that.

3 RESEARCH QUESTIONS AND DESIGN OBJECTIVES

3.1 The two motivating goals

The framework is designed against two goals, stated by the originating practitioner and retained unchanged because they are the purpose rather than an intermediate means.

```
G1 (Comprehension support). To enable a practitioner using an AI coding
   assistant -- including one without formal training in the domain -- to
   build an accurate, evidence-backed understanding of an unfamiliar domain,
   using systematic external research rather than the assistant's priors.


G2 (Problem discovery). To enable that practitioner, having reached the
   comprehension of a domain competent practitioner, to identify problems
   that the domain's own actors have not articulated -- because they are
   structural, latent, or outside the frame within which the actors think --
   and to formulate technology-mediated responses to them.
```

These goals are sequential and are not collapsed. G1 produces a validated domain model. G2 operates *on* that model, and its output is a set of falsifiable hypotheses, not a feature list. G3

(not stated by the practitioner but implied by the deliverable) is to produce a specification and implementation whose every element is traceable to G1 or G2.

The relationship is:

```
G1: What is true of this domain?
G2: Given what is true, what should be questioned?
G3: Given what should be questioned, what exactly will be built, and can we
    demonstrate that what was built does that?
```

3.2 Research questions

```
RQ1. Can a machine-checkable artifact model, in which epistemic state is part
     of the data, make it possible for an autonomous agent to preserve
     uncertainty through an entire engineering lifecycle rather than
     collapsing it into confident prose?


RQ2. Does requiring a structured derivation chain between external evidence
     and requirement reduce the rate of unfounded requirements in agent-
     authored specifications?


RQ3. What is the minimum set of mechanically checkable gate conditions that
     detects epistemic laundering (the progressive promotion of ASSUMED or
     INFERRED knowledge to VALIDATED) and evidence-free requirements?


RQ4. Can the addition of a mandatory, hypothesis-governed opportunity stage
     reduce feature replication -- the reproduction of competitor capability
     presented as analysis -- in agent-generated specifications?


RQ5. Does a formal rejection of MVP-as-scope-device, replaced by
     evidence-bounded capability classification, improve the long-run
     coherence of agent-generated systems relative to MVP-scoped systems?


RQ6. Is the framework's overhead tolerable? That is: is the marginal cost of
     evidence and traceability maintenance lower than the expected cost of
     semantic fabrication, at the granularity of a single change?
```

RQ1-RQ3 are primarily design questions answered by construction and evaluation protocol. RQ4-RQ6 are empirical and are stated here as open.

3.3 Non-goals

- OpenDomain does not assert that AI coding assistants are net harmful. It asserts that they amplify the cost of an already-existing epistemic gap.
- OpenDomain does not replace expert judgement. Where an expert is available, the framework's cost is close to zero; the marginal value is in recording and enforcing the expert's own traceability.
- OpenDomain does not adopt microservices, event sourcing, CQRS, or any other architectural style. Architecture is derived from domain boundaries and quality attributes, never from fashion (Section 5.9).
- OpenDomain does not aim to maximise the number of artifacts. It enforces a minimal-artifact principle (Section 5.11): an artifact exists only if it provides clarity, traceability, verification, coordination, a decision record, or maintenance value.

4 BACKGROUND AND RELATED WORK

4.1 Domain knowledge in requirements engineering

Araujo et al. [1] report a systematic mapping study of domain knowledge in requirements engineering, screening 75 papers against inclusion criteria. Three findings of that study are load-bearing for this work.

First, domain knowledge is established as a key component of RE: it supplies the conceptual support needed to understand system context, align with stakeholder needs, and reduce ambiguity in specification. This paper does not contest that finding; it treats it as the premise.

Second, the study identifies recurring challenges in the *formalisation*, *acquisition*, and *long-term maintenance* of domain knowledge. This is precisely where the agentic setting bites hardest. Acquisition, in the practitioner's absence of experts, becomes a research activity; formalisation becomes a language-design activity; and maintenance becomes an impact-analysis activity. All three are activities that a human performs slowly and an agent performs fast and badly.

Third, the study's stated future work emphasises "scalable, automated, and sustainable solutions to integrate domain knowledge into RE processes." The scalability qualifier is the crux. Most existing domain-knowledge approaches are project-scale activities performed by a team over weeks. An agent operates at change-scale, continuously, without a team. A method optimised for the former regime cannot be simply transferred to the latter.

Our position: the sustainability problem named by [1] is an engineering problem with an engineering answer. It is solved by making knowledge artifacts version-controlled, incrementally maintained, and machine-checked – not by adding ceremony.

4.2 Specification-driven development

Piskala [2] provides a practitioner's guide to specification-driven development (SDD), inverting the traditional workflow so that specifications rather than code are the primary artifact. Three elements of that work are adopted directly.

The first is the tiered notion of specification authority: spec-first (specification then code), spec-anchored (specification and code co-evolve under mutual constraint, the default for production systems), and spec-as-source (specification generates code, applicable only where generation tooling is trustworthy). We adopt this tiering because it correctly identifies that specification strictness is a function of consequence and duration, not of methodological taste. We note explicitly the paper's own warning that LLM-generated application code should not be treated as spec-as-source: an agent's generation is a proposal requiring verification, not a derivation with guaranteed correctness.

The second is the separation of stages – specification, plan, task, implementation – which the paper treats as mandatory. This paper's contribution is to extend that chain *upstream* of specification, to the evidence that justifies the requirements the specification consumes, and *laterally*, with an opportunity stage that the SDD literature does not include.

The third is the paper's identification of false confidence as a major pitfall: software can faithfully implement an incorrect specification, so passing tests do not establish correctness. This paper accepts the observation and asks the question the paper leaves open: how is the *correctness of the specification itself* established, given that the specification is frequently agent-authored? Our answer is that it cannot be established by the agent, and that the correct system behaviour is to record the absence of establishment rather than to imply it.

4.3 Domain-specific modelling languages and MBSE

Beers et al. [3] address a closely adjacent problem for production systems. Their work introduces a domain-specific modelling language (DSML) for the conceptual design phase, built on the SPES

(Software Platform Embedded Systems) framework and extending SysML through the UML profile mechanism, with viewpoint-specific diagram types and automated consistency checks. To address domain anchoring, they develop five supplementary DSMLs derived from established industrial standards (VDI/VDE 3682, OPC UA Robotics, IEC 61293, TCP/IP, ISO 10303).

This work is instructive in two ways, and we align with it on both.

First, it isolates the same diagnosis: approaches that are methodologically sound but *not anchored in domain-specific knowledge* are of limited practical applicability. That is our Section 2 thesis, arrived at from a different direction. The evidence is not the analyst's failing to be rigorous; it is the absence of a knowledge substrate to be rigorous about.

Second, and importantly for how we position OpenDomain, their evaluation identifies a residual gap: while the base language addresses clear representation and traceability, "domain anchoring through standards" is not yet addressed in the base, and requires the supplementary standard-based languages to achieve. We read this as evidence that domain knowledge is not a property of a modelling notation. Notation does not carry it. It has to be carried by the knowledge artifacts themselves, which is a different mechanism from the one [3] pursues. The two are complementary: OpenDomain supplies the evidence and derivation substrate; a DSML such as RFLT can supply the notation into which the resulting domain model is projected.

We differ in scope. [3] is a systems-engineering contribution for a specific class of industrial system, with a fixed notation and a case-study evaluation. OpenDomain is notation-agnostic and domain-agnostic, and its output is artifacts and gates rather than a diagram language.

4.4 Domain-driven design

Domain-Driven Design [4] supplies the strategic core – bounded contexts, the distinction between a domain model and a data model, the imperative to put behaviour in the domain – and the anemic-domain-model critique [7] supplies the diagnostic that behaviour living in services rather than entities produces transaction scripts, not domain models.

OpenDomain adopts the strategic core. It departs from DDD in one respect that matters here: DDD presupposes a team with access to domain experts, and its practical failure mode under AI assistance is the reverse of its thesis. Because a generative model produces class hierarchies cheaply and fluently, an agent applied to an insufficiently understood domain will produce an *anemic-looking but behaviourally-empty* model with high surface plausibility: many entities, aggregates, and repositories, with the actual rules nowhere. DDD's warning about anemic models is, in the agentic setting, better stated as: the model is not anemic, it is *deficient*, and the deficiency is invisible because the structure is present.

This is the general form of the problem in Section 1.2: structure presence masks semantic absence.

4.5 Specification by example, BDD and TDD

Specification by Example [10] establishes that concrete examples can serve as the authoritative specification, and BDD formalises this into a human-readable-yet-executable form (Gherkin [25]). TDD [11] applies the same discipline one level down, at unit granularity. xUnit Test Patterns [12] supplies the taxonomy of test doubles and their correct application.

These are adopted, with one significant qualification discussed in Section 9: BDD, TDD, contract testing, property-based testing and model-based specification are *different verification mechanisms with different applicability conditions*, and a framework that mandates one globally is prescribing a style rather than engineering a guarantee.

4.6 The Minimum Viable Product and its limits

The MVP concept [13, 14] addresses a real and important problem: the coupling of scope to learning. Ship the smallest thing that produces the most learning per unit of effort, and let the feedback loop correct the plan. This is a significant advance on plan-driven sequential delivery.

Its applicability conditions are, however, narrower than its general use. Section 8 analyses these conditions and argues that they are violated in the OpenDomain target setting. Section 8 is not a criticism of MVP; it is a scope decision, argued and recorded, in the spirit of the framework itself.

4.7 Position summary

Work	Knowledge substrate	Verifiability of knowledge	Agent-safe by default	Scope
Araujo et al. [1] (SMS)	yes	n/a	no	project
Piskala [2] (SDD)	inherited	no	partial	project
Beers et al. [3] (RFLT DSML)	yes (std)	partial (rules)	no	domain
DDD [4, 7]	tacit	no	no	team
BDD / TDD	Section 9.2; Section 9.7			
Boehm [13] / MVP [14]	business	n/a	n/a	product
OpenDomain (this work)	yes	yes (gates)	yes	change

The gap this work addresses is the intersection of the "knowledge substrate" column with the "verifiability of knowledge" and "agent-safe by default" columns. No reviewed work populates all three.

4.8 Mapping from founding operational protocols

OpenDomain is operationalised through three protocol documents that function as its executable specification: a domain-engineering protocol (two variants, a practical and a knowledge-driven), a spec-driven-engineering protocol, and an opportunity/innovation-engineering protocol. These are the material from which the framework is abstracted; Appendix D gives the full mapping. Two design rules govern that abstraction and are stated here because they explain the framework's shape:

```
Rule A (do not invent). A protocol instruction that requires information the
agent may not possess must degrade to an explicit UNKNOWN or CONFLICTING
record, never to a plausible default.
```

```
Rule B (mechanise the prohibition). Any protocol instruction phrased as
"never do X" must be expressible as a check the tooling can execute, or it
will be honoured in practice only by authors who already honour it in
principle. Most of the prohibitions in the founding protocols are of the
unmechanised kind; Section 7 converts the tractable ones into gates.
```

5 THE OPENDOMAIN FRAMEWORK

5.1 The pipeline

OpenDomain organises engineering as a nine-gate pipeline. Each gate has a precondition (what must hold to proceed) and produces artifacts consumed by the next.

```
Gate 1  context        Establish what the project is, for whom, and under
                       what constraints. No modification of code.
Gate 2  research       Acquire and register evidence; detect saturation and
```

```
                    conflicts. Terminate research explicitly.
Gate 3  domain      Actors, stakeholders, entities, events, processes,
                    rules, states, ownership, cross-domain relations.
Gate 4  requirements Derive requirements through the evidence gate; classify
                    and challenge; non-functional requirements in the
                    same pass; establish a baseline.
Gate 5  opportunity Reconstruct the current state, extract assumptions,
                    find contradictions, generate and attack concepts,
                    formulate hypotheses, design experiments, converge.
Gate 6  specification Behavioural specification, acceptance criteria,
                    error and boundary scenarios, contracts.
Gate 7  plan        Architecture, data model, interfaces, decisions, task
                    decomposition, dependency ordering.
Gate 8  implementation Incremental, vertical-slice, test-first, with drift
                    checks between increment and specification.
Gate 9  completion  Full alignment audit: knowledge -> requirement ->
                    specification -> plan -> code -> test, plus
                    simplification and cross-module audits.
```

Gate 5 is placed *after* requirements rather than before domain for a reason worth stating, since it is counter-intuitive. Opportunity analysis performed before the domain is understood produces feature generation. Performed after requirements exist, it operates on a validated model of current reality and can therefore distinguish "this is what the domain does" from "this is what we believe could be different". Ordering it after Gate 4 also means the framework does not require a project to justify its existence through innovation, which matters for regulated and internal systems whose scope is externally determined. Gate 5 may terminate in "no material opportunity identified", which is a legitimate and recorded outcome.

5.2 The epistemic lattice

Every artifact carries a knowledge-type value drawn from a closed set of seven:

```
EXPLICIT    Directly documented in an authoritative source: regulation,
            standard, official documentation, textbook, peer-reviewed
            publication, published process description.
TACIT       Held in human expertise or organisational practice but not
            documented. Real, load-bearing, and not transferable by
            reading.
HYBRID      Reconstructed by combining explicit sources with expert or
            operational reasoning. The reasoning must be recorded.
INFERRED    Derived by the agent from multiple evidence items. The
            derivation must be recorded.
ASSUMED     A temporary position adopted to permit progress. Must be
            recorded with its expiry condition.
UNKNOWN     Evidence insufficient. No answer recorded.
CONFLICTING Reliable sources disagree. Both positions recorded; a resolution
            status is required.
```

Three properties are deliberate and consequential.

First, the set is closed and has no default. A new artifact without a declared knowledge type is invalid. This is what makes ignorance representable: the frame does not permit an artifact to exist in an undeclared epistemic state, which is the state in which fabrication is invisible.

Second, the lattice is not ordered in the way confidence usually is. TACIT is not "less" than EXPLICIT; it is differently located, and often more decision- relevant, since undocumented operational heuristics frequently govern what actually happens while the documented procedure describes what is supposed to happen. The framework records both and marks the divergence rather than resolving it silently. The related distinction between historically necessary and fundamentally necessary steps is treated in Gate 5 as an assumption to be challenged, not as a modelling decision to be made silently.

Third, epistemic state is per-artifact and mutable, and the lattice does not permit monotonic promotion. An artifact may move INFERRED → EXPLICIT when a source is found, or ASSUMED → CONFLICTING when a contradicting source appears, or any state → UNKNOWN when a source is retracted. Gate 8 detects the specific prohibited transition: promotion of ASSUMED or INFERRED to VALIDATED without an intervening EXPLICIT or corroborating evidence. This is the mechanisation of "epistemic laundering", and it is, in our assessment, the single most valuable check in the framework.

5.3 The evidence gate on derivation

The framework forbids the direct promotion of an external statement into a requirement. The permitted path is:

```
Source
  -> Finding          what the source literally states
  -> Interpretation   what we read that to mean, in our situation
  -> Candidate Need   a need this might serve
  -> Requirement      a SHALL statement derived from a candidate need
```

The value of the intermediate steps is not ceremonial. Consider the most common concrete failure in agent-generated requirements: a competitor publishes a product page stating that its inventory system supports "reservation". The failure is the jump from that page to "REQ: The system SHALL support stock reservation."

```
Source        Competitor product page, retrieved DATE.
Finding       The page states that reservation is supported.
Interpretation The page does not state *why*, does not state whether
               reservation is legally required or commercially optional,
               and does not state the lifecycle position of reservation
               relative to payment.
Candidate     Possibly the deployment organisation experiences stock loss
              from concurrent allocation of the same units.
Requirement   Blocked. The candidate need is a hypothesis; it requires
              domain evidence or explicit business confirmation before it
              can become a requirement.
```

The gate converts an illegitimate step into a visible, closable question. That is the entire mechanism.

5.4 The traceability spine

OpenDomain maintains one ordered chain:

```
Source -> Evidence -> Domain Knowledge -> Requirement -> Use Case
  -> Specification -> Acceptance Criterion -> Contract -> Plan -> Task
  -> Code -> Test -> Validation Result
```

over a fixed relation vocabulary of thirteen relations:

```
supports        A rests on evidential support from B
derived-from    A was extracted from B
implements      A realises B
depends-on      A cannot exist without B
constrains      A narrows the permitted implementations of B
verifies        A checks that B holds
supersedes      A replaces B
conflicts-with  A and B cannot both hold as written
affected-by     A is downstream of B and must be revisited when B changes
```

```
owned-by        A is the single owning domain of B's concept
contains        A subsumes B into its own lifecycle
answers         A answers B's question
tests           A tests B's falsifiable claim
```

The relation vocabulary is closed. This matters: an open relation vocabulary degenerates into a graph of undocumented semantics in which "derived-from" and "related-to" become indistinguishable, and impact analysis becomes guesswork.

One relation is reserved by construction. The binding edge from a verification artifact to the acceptance criterion it checks is expressed as tests, not as verifies, because the direction of the edge carries the epistemic claim: a test asserts a falsifiable proposition, whereas verifies would assert that the proposition is already established. The provisioner emits that edge, and never infers it (Section 10.8).

The spine supports two query classes that define the framework's operational value proposition:

```
Q-Why    For artifact A, traverse backwards along supports / derived-from /
         implements until a source with an authority rating is reached.
         The answer to "Why does this code exist?" is then a path, not a
         recollection.
Q-Blast  For artifact A, traverse forwards along implements / depends-on /
         affected-by / verifies to enumerate every artifact invalidated by a
         change to A.
```

Both queries are graph traversals over a version-controlled file set. Both are answerable by an agent without interpretation. This is, we argue, the property that most directly determines whether an agent can maintain a large system coherently: not the quality of any single generation, but the ability to enumerate the consequences of a change.

5.5 Bidirectionality and the pre-requirements question

Traceability literature distinguishes requirements traceability (forward, from requirement to implementation) from pre-requirements traceability (backward, from requirement to the stakeholder information, documentation, or legacy system that introduced it). The agentic setting makes the backward direction the binding constraint, because it is exactly the direction in which an agent generates content without provenance: the forward direction is recoverable from the artefacts themselves, the backward direction is not.

OpenDomain therefore treats the source registry as a first-class artifact with a mandatory authority rating and access date, and treats a requirement without a resolvable evidence path as a gate failure rather than as an incomplete document. Pre-requirements traceability is the mechanism by which the framework distinguishes "the organisation needs this" from "a model inferred it".

5.6 Cross-domain analysis and concept ownership

For systems with more than one module, the framework requires that every significant shared concept have exactly one owning bounded context, and that cross-context references be explicit relationships rather than redefinitions. The specific failure prevented is concept fragmentation: Sales.Customer, Inventory.Customer, Procurement.Customer existing as three incompatible records for one real-world party.

The framework also requires, for every capability, an explicit statement of what it produces, consumes, reads, writes, triggers, and depends on, and requires that a new module never be analysed in isolation. Ownership is recorded as an `owned-by` relation, making the single-owner constraint checkable.

5.7 The opportunity stage

Gate 5 exists to prevent a specific and highly probable agent failure: given a competitor's feature list and a domain description, the agent will produce a feature list. This is feature replication, and it is presented in the register of analysis.

The stage's structure is designed to make replication structurally visible:

```
1. Current-state reconstruction. Represent how the problem is solved today as
   Actor -> Action -> Information -> Decision -> Action -> Outcome.
2. Friction discovery. Search beyond stated feature requests: pain, friction,
   waste, delay, repetition, duplication, context switching, information
   asymmetry, decision bottlenecks, error-prone operations, invisible work,
   unmet needs users do not state because they assume them impossible.
3. Assumption extraction. For each significant step, ask why it exists and
   whether the creating constraint still holds. Record ASSUMED artifacts.
4. Contradiction analysis. Identify places where current behaviour and
   desired behaviour conflict, and the assumption underlying each.
5. Opportunity spaces. Convert problems into spaces of possible direction,
   not into features.
6. Divergent exploration. Apply a fixed operator set (remove, automate,
   delegate, merge, split, invert, predict, proact, personalise, autonomise,
   combine, abstract, reframe, reverse) and generate at least three
   materially different concepts, not three variants of one.
7. Concept critique. Attack each concept on necessity, novelty, value,
   feasibility, evidence, risk, adoption, complexity, defensibility,
   unintended consequence, and failure mode.
8. Hypothesis formulation. Convert the surviving concepts into falsifiable
   hypotheses with target population, metric, threshold, experiment, and
   failure condition.
9. Experiment design. Prefer the cheapest meaningful experiment: interview,
   Wizard-of-Oz, synthetic-data test, technical spike, concierge, small
   vertical slice. This is the mechanism by which a candidate is killed
   before implementation.
10. Convergence. A concept is frozen only against an explicit checklist; the
    outcome is a Concept Decision Record stating why this concept is being
    pursued, not that it is the best concept.
```

A named anti-pattern check ("innovation red flag") is applied: if the proposal is substantially a competitor feature list, justified by "everybody does this", identifying no underlying problem, challenging no assumption, exploring no alternative workflow, and proposing no new mechanism, the stage returns to assumptions and contradictions rather than proceeding.

Two outputs are explicitly recorded as legitimate: "no material opportunity identified", and "hypothesis failed; pivot recorded with the surviving knowledge". Failure is retained as an artifact rather than deleted, because deleted failures are re-derived.

5.8 Uncertainty as a managed, not suppressed, quantity

The framework maintains a first-class uncertainty registry. Every entry records the question, why it is unknown, the evidence available, the possible answers, the impact of each, the confidence, and the validation required. Two disciplines apply.

The first is that uncertainty may be closed by research but not by preference. Where external evidence can answer a question, the agent researches rather than asks. The agent asks the human only when the decision is project-specific, when the answer cannot responsibly be researched, when materially different interpretations have materially different consequences, or when authorisation is required. When asking, the framework requires the agent to state the question, why it matters, the known alternatives, the impact of each, and (on request) a recommendation. The intent is to preserve the human's scarce attention for the decisions only a human can make, which in practice means business and organisational decisions, not vocabulary questions.

The second is that regional and organisational context is evidence-requiring. Generic international workflow is not a substitute for local practice; local regulation, terminology, payment systems, administrative procedure, taxation, reporting requirements, and organisational structure each require their own evidence.

5.9 Architecture is derived, not selected

Architecture follows the domain. The framework requires that architectural choices be justified by domain boundaries, consistency requirements, scale, security, deployment constraints, data ownership, team constraints, and existing technology – and explicitly not by fashion. Microservices, event sourcing, CQRS, message brokers, plugin systems, generic frameworks, and distributed caches are listed as constructs requiring a concrete justifying requirement. The default posture is the simplest architecture that satisfies the validated requirements, which for many systems is a modular monolith.

The rationale is not conservatism; it is that each distributed construct transfers a solution (a distributed transaction) into a distributed set of failure modes, and that transfer must be paid for deliberately. A significant share of agent-generated architectures fails here, because distribution is visually attractive in a generated dependency graph and its costs are not visible in one.

5.10 Brownfield: existing code is evidence, not specification

For existing systems the framework requires reverse specification with explicit behaviour classification. Observed behaviour is classified as documented, inferred, observed, legacy-accidental, intentional, compatibility-preserving, or unknown. Nothing is deleted on the grounds of being undocumented, and nothing is preserved on the grounds of being implemented. Classification precedes change.

This is the direct brownfield analogue of the evidence gate, and it prevents the common agent error of treating the current implementation as the requirement.

5.11 The minimal-artifact principle

The framework enumerates document types generously and then requires that each be justified by at least one of: clarity, traceability, verification, coordination, decision record, maintenance value. Documents that provide none of these are to be merged or deleted. This is the counterweight to the framework's own proliferation: a method that generates forty documents for a two-week change will be abandoned, and abandonment returns the process to the unstructured state it was meant to replace.

Where the reference implementation scaffolds a full document set, the scaffolding is a menu, not a mandate.

5.12 The framework's own audit

The framework is subject to its own requirements. This section states its epistemic status precisely, because doing so is the framework's first test of itself.

- The failure analysis (Section 1.2) is INFERRED from observed agent behaviour, not from a controlled measurement.
- The claim that structure presence masks semantic absence is INFERRED.
- The MVP rejection (Section 8) is an argument, not a measurement. The empirical claim is OPEN.
- The gate set (Section 7) is designed, untested for coverage and false-positive rate.
- The reference implementation (Section 11) is a prototype. It does not currently build in the authoring environment; see Section 11.6.

- No claim of improved software quality, reduced defect rate, or reduced rework is made anywhere in this paper.

6 ARTIFACT MODEL AND IDENTIFIER SYSTEM

6.1 Identifier kinds

The reference implementation defines 37 identifier kinds. The grouping below is the framework's, not the code's.

```
Knowledge substrate  SRC source, EVD evidence, RES research question,
                     TERM glossary term, CONFLICT source conflict,
                     DEC decision
Domain model         DOM domain, STK stakeholder, ACT actor, PROC process,
                     ENT entity, EVT event, RULE business rule,
                     INT integration, UNC uncertainty
Opportunity          OPP opportunity, ASM assumption, CONTRAD contradiction,
                     CPT concept, HYP hypothesis, EXP experiment,
                     CDR concept decision record
Requirements         REQ requirement, NFR non-functional requirement,
                     CNST constraint, BAS baseline, UC use case
Specification        SPEC specification, AC acceptance criterion,
                     CTR contract, SCN scenario
Delivery             PLAN plan, TASK task, ADR architectural decision record,
                     TEST test, GAP gap, DRIFT drift record
```

Two properties are load-bearing.

Stable typed prefixes. Because the prefix encodes the kind, a link `derived-from: ["RULE-001"]` is checkable by inspection, and a requirement that cites a test is a distinct, detectable condition from one that cites a rule. The alternative – opaque UUIDs with typed edges – makes the most valuable class of error (a requirement citing a test as its justification) invisible.

Scoped identifiers. Where a system has multiple bounded contexts, identifiers may be scoped (`REQ-INVENTORY-012`, `AC-INVENTORY-012-01`), and acceptance criteria inherit the scope of their requirement. This makes cross-context traceability and cross-context *collision* both detectable, supporting the single-owner constraint of Section 5.6.

6.2 Artifact statuses

Thirteen statuses, in lifecycle order:

```
Draft, Researched, Supported, Validated, Approved, Implementing,
Implemented, Verified, Blocked, Open, Conflicting, Deprecated, Superseded
```

The ordering is a partial order over epistemic and lifecycle state jointly, and the two are not the same axis. VALIDATED is an epistemic claim; IMPLEMENTING is a lifecycle claim. The framework's core discipline is that an artifact may be IMPLEMENTED while its epistemic status remains ASSUMED, and the gate set checks this specifically: implementation progress does not launder epistemic state.

6.3 Artifact record structure

Every artifact is a Markdown document with a heading carrying the identifier and a fenced TOML block carrying structured fields, followed by prose. The split matters: structured fields are machine-checked; prose is human-read. The reference implementation parses the heading and the block, and treats the prose as uninterpreted content, which means the framework does not require

natural language to be parseable – an important property given that one of its primary producers is a language model.

The canonical field set is:

```
id, kind, title, status, epistemic, confidence, authority,
owner, scope, created, updated, version,
supports[], derived-from[], implements[], depends-on[], constrains[],
verifies[], supersedes[], conflicts-with[], affected-by[], owned-by[],
contains[], answers[],
body (free prose)
```

Appendix A gives per-kind field requirements.

6.4 Authority and confidence

Two orthogonal scalar attributes distinguish the strength of support from the familiarity of the claim.

```
Authority  how binding the source is *for this particular claim*. A
           manufacturer is authoritative for its own product; a regulator is
           authoritative for compliance; a peer-reviewed paper is
           authoritative for a research claim; an experienced practitioner may
           be authoritative for tacit operational knowledge. Authority is
           claim-relative, not source-relative, and the framework records
           authority per evidence item rather than per source.

Confidence  HIGH, MEDIUM, LOW, applied to the inference, not to the source.
           Confidence is reduced, not preserved, when multiple derived
           sources rest on one original assertion; the framework explicitly
           refuses to inflate confidence from repetition.
```

7 THE NINE QUALITY GATES

7.1 Design constraint

Gates are pure functions from a workspace index to a report. They do not mutate state. Two consequences follow. First, gate evaluation is reproducible: running it twice yields the same result, which is what makes it trustworthy enough to block. Second, gates are auditable as code: the exact condition that failed is nameable, which is what makes remediation possible. A gate that both mutates and judges cannot be trusted in either role.

Severity has three levels. BLOCKER prevents progression. WARNING requires an explicit recorded decision to proceed. NOTE is advisory.

7.2 Gate 1 – context

Precondition: none. Output: project context, stakeholders, constraints, classification of the project as greenfield, brownfield, or hybrid, and an explicit statement of known problems and known unknowns.

Rule	Severity	Condition
G1-B1	BLOCKER	No project purpose recorded.
G1-B2	BLOCKER	Project not classified greenfield/brownfield/hybrid.
G1-B3	WARNING	Repository inspected? (brownfield only; absence suggests the gate was satisfied from assumption)
G1-W1	NOTE	No known constraints recorded.

Rationale: the majority of wasted agent effort is spent on the wrong project. This gate is cheap and prevents the most expensive error class.

7.3 Gate 2 – research

Precondition: G1 passed. Output: source registry, evidence registry, search strategy, research log, glossary, uncertainty and conflict registries.

Rule	Severity	Condition
G2-B1	BLOCKER	Any evidence item has no resolvable source.
G2-B2	BLOCKER	Any EVIDENCE item declares epistemic = EXPLICIT while its source is absent, undated, or unrated.
G2-B3	BLOCKER	Any registry artifact has no declared epistemic value.
G2-B4	BLOCKER	Research declared saturated while a required category (actors, processes, entities, rules, exceptions, constraints, integrations, NFRs, risks, regulation, alternatives, contradictions) is empty.
G2-W1	WARNING	Any CONFLICTING artifact with no resolution status.
G2-W2	WARNING	Any important claim resting on a single source.
G2-W3	WARNING	Any significant term with no canonical definition, or one term carrying two meanings.
G2-N1	NOTE	Research log does not record a termination reason.

G2-B4 deserves emphasis. The failing condition is not "insufficient research" in the general sense, which is undecidable, but "a declared completion while a named category is empty". This converts an undecidable judgement into a checkable one, and it is the difference between research saturation as a self-assessment and research saturation as a verified property.

7.4 Gate 3 – domain

Precondition: G2 passed. Output: domain map, glossary, actors, stakeholders, entities, relationships, events, processes, rules, states, integrations, ownership assignments.

Rule	Severity	Condition
G3-B1	BLOCKER	A process has no identified trigger.
G3-B2	BLOCKER	A business rule exists with no source and no explicit ASSUMED marking.
G3-B3	BLOCKER	Two artifacts claim ownership of the same concept within one scope.
G3-B4	BLOCKER	An entity is modelled with no identified lifecycle behaviour, no business rule governing it, and no requirement referencing it. (An entity introduced by convention.)
G3-W1	WARNING	A process models only a happy path (no exception or alternative scenario).
G3-W2	WARNING	A state with no identified transition trigger.
G3-W3	WARNING	A capability produces or consumes a cross-domain artifact with no recorded owner.
G3-N1	NOTE	No event recorded for a state change that occurs in a multi-actor process.

G3-B4 is the single most useful domain gate. It is the mechanical form of "do not create classes because nouns appear in requirements". Its evidence is a conjunction: a concept must have lifecycle behaviour, a governing rule, or a referencing requirement. A concept with only a name is not a domain concept.

7.5 Gate 4 – requirements

Precondition: G3 passed. Output: requirements, non-functional requirements, constraints, requirement baseline, requirement-to-rule and requirement-to- evidence links.

Rule	Severity	Condition
G4-B1	**BLOCKER**	A requirement has no derivation path to evidence, a rule, or an explicit ASSUMED marking.
G4-B2	**BLOCKER**	A requirement's epistemic state is EXPLICIT but its evidence resolves to fewer than one source, or to a source not rated authoritative for the claim.
G4-B3	**BLOCKER**	A requirement is not atomic (contains two or more SHALL obligations that could be satisfied or failed separately).
G4-B4	**BLOCKER**	A requirement is untestable (contains no observable criterion).
G4-B5	**BLOCKER**	A functional area has no non-functional requirements.
G4-B6	**BLOCKER**	A baseline exists that includes an artifact whose epistemic state is UNKNOWN or CONFLICTING without recording it as an exclusion or open question.
G4-W1	**WARNING**	A requirement cites a competitor source and no domain need.
G4-W2	**WARNING**	Two requirements are mutually unsatisfiable.
G4-W3	**WARNING**	A requirement contains an unfalsifiable qualifier (fast, intuitive, robust, scalable, user-friendly, intelligent).
G4-W4	**WARNING**	No requirement references any exception or failure path.
G4-N1	NOTE	A requirement has no acceptance criteria.

G4-B1 is the evidence gate mechanised. G4-W3 is the "intuitive/fast" gate: the framework's requirement-quality rules reject vague requirements, and this check makes the rejection mechanical for the most common vocabulary of vagueness.

7.6 Gate 5 – opportunity

Precondition: G4 passed. Output: current-state reconstruction, friction map, assumption registry, contradiction registry, opportunities, concepts, hypotheses, experiments, concept decision record.

Rule	Severity	Condition
G5-B1	**BLOCKER**	A specification advanced past Gate 5 with no opportunity record. Either no opportunity was considered, or the consideration was not recorded. Both are defects; the gate does not require an opportunity to have been found.
G5-B2	**BLOCKER**	A concept advanced to specification with no hypothesis stating a falsification condition.
G5-B3	**BLOCKER**	A Concept Decision Record is absent.
G5-W1	**WARNING**	A hypothesis has no threshold or no experiment.
G5-W2	**WARNING**	A concept is justified substantially by competitor feature presence with no stated domain problem. (Innovation red flag.)
G5-W3	**WARNING**	An experiment result of INCONCLUSIVE was recorded as VALIDATED.
G5-N1	NOTE	Fewer than three materially different concepts were considered for a significant opportunity.

G5-B1 is the most structurally important gate in the framework, and the one most likely to be resisted. It requires a *record*, not an *innovation*. A team building a mandated compliance feature records "no material opportunity; scope externally determined", and proceeds. What it may not do is skip the stage, because skipping is indistinguishable in the output from having skipped the

thinking. G5-W3 protects against the specific and common error of converting an inconclusive experiment into a validation, which is the experimental analogue of epistemic laundering.

7.7 Gate 6 – specification

Precondition: G5 passed. Output: specifications, acceptance criteria, contracts (API, event, data, state, security, non-functional), scenarios.

Rule	Severity	Condition
G6-B1	**BLOCKER**	A specification has no linked requirement.
G6-B2	**BLOCKER**	A specification prescribes implementation where the requirement does not (binds technology not mandated by requirement or constraint).
G6-B3	**BLOCKER**	A specification has no error or boundary scenario for a behaviour with a failure mode.
G6-B4	**BLOCKER**	An acceptance criterion is not measurable.
G6-B5	**BLOCKER**	An API or event contract exists with no stated authorisation model, or with a breaking change not reflected in a version increment.
G6-B6	**BLOCKER**	A behaviour with a specified verification rigour has no executable form: an acceptance criterion that declares a BDD or contract rigour has no scenario and no contract, and one that declares a unit rigour has no binding test. (Rigour declaration without an artifact is a declaration; see Section 9.7 and Section 10.8.)
G6-W1	**WARNING**	A specification has no alternative scenario.
G6-W2	**WARNING**	A specification mixes behavioural requirement with technical design in the same artifact.
G6-W3	**WARNING**	A declared rigour is satisfiable only by a framework the project has not adopted, and no adoption decision is recorded.
G6-N1	NOTE	A specification is DRAFT while its implementing tasks exist.

G6-B2 enforces the what-before-how separation, and is the check that prevents the most common specification defect: a specification that has quietly decided the architecture, leaving no room for the technical plan to make an evidence-based choice.

G6-B6 is what makes Section 9 falsifiable. A rigour column filled in by intuition produces a table that looks like discipline and contains no commitment; requiring the artifact to exist is the only check that distinguishes the two. The gate does not require a particular framework, only that the declared rigour have an executable instance.

7.8 Gate 7 – plan

Precondition: G6 passed. Output: architecture, component map, data model, interface contracts, ADRs, task decomposition, dependency graph, testing strategy.

Rule	Severity	Condition
G7-B1	BLOCKER	A task has no linked specification or requirement.
G7-B2	BLOCKER	Two parallel tasks modify the same architectural boundary without a coordination record.
G7-B3	BLOCKER	A component has no requirement justifying its existence.
G7-B4	BLOCKER	An architectural decision has no ADR.
G7-W1	WARNING	A distribution construct (microservice boundary, event bus, CQRS split, distributed cache) with no justifying requirement or quality attribute.
G7-W2	WARNING	No failure-mode consideration for an external integration.
G7-W3	WARNING	A data model entity has no mapped domain concept.
G7-N1	NOTE	No migration strategy for a brownfield change.

G7-B3 is the anti-overengineering gate in its most direct form. Its asymmetry is deliberate and worth defending: orphans on the *requirements* side (a requirement with no implementation) are BLOCKERs elsewhere (G8/G9), and orphans on the *component* side are BLOCKERs here. The framework is symmetric in its indignation.

7.9 Gate 8 – implementation

Precondition: G7 passed. Output: code, tests, incremental validation records, drift records.

Rule	Severity	Condition
G8-B1	BLOCKER	A link references a non-existent target. (Dangling reference.)
G8-B2	BLOCKER	An artifact's epistemic state is ASSUMED or INFERRED and its status is VALIDATED or VERIFIED, with no intervening EXPLICIT evidence. (Epistemic laundering.)
G8-B3	BLOCKER	A test exists with no specification, acceptance criterion, or requirement reference.
G8-B4	BLOCKER	An acceptance criterion has no test.
G8-B5	BLOCKER	A test exists whose recorded rigour is higher than the rigour the suite can actually demonstrate, or whose recorded verification depth is executed but whose last recorded run is red. (Claimed-versus-actual verification; see Section 10.9.)
G8-B6	BLOCKER	A verification artifact declares binding = INFERRED to an acceptance criterion or rule, where the declared binding is the sole justification for that criterion passing. (An unconfirmed binding cannot close a traceability edge.)
G8-B7	BLOCKER	A verification toolchain was provisioned by the framework and its versions, the selection rule that chose it, and the install diff are not recorded as a decision artifact. (An undocumented framework change is an undocumented architectural change; see Section 10.2, PP6.)
G8-W1	WARNING	A requirement is IMPLEMENTED with no test.
G8-W2	WARNING	A code change affects a concept with recorded owners in more than one scope, with no impact record.
G8-W3	WARNING	A specification and an API contract disagree on a constraint (the drift case: spec says max 100, API accepts 1000).
G8-W4	WARNING	A scenario or step definition exists that no acceptance criterion cites (the reverse of G8-B4; orphan verification).
G8-N1	NOTE	A behaviour described in a specification has no sequence or state artifact.

G8-B2 is the laundering check. Its practical form: the framework requires that the transition

ASSUMED/INFERRED → VALIDATED be accompanied by a new evidence item, not merely by a status edit. This is trivially cheap to check and, in our assessment, the highest-value single check in the framework. A status change without a provenance change is the signature of laundering.

G8-B5, B6 and B7 are the checks that the verification subsystem cannot itself launder. Each of the three admits a specific way for the framework to claim coverage it does not have: a rigour column written by hand and never executed (B5), a binding edge guessed from a name match and treated as traceability (B6), and a test framework adopted without a decision record (B7). B7 matters because the provisioner is the largest single source of automatic architectural change the framework makes, and an unreviewed dependency added to a build is a supply-chain event (Section 10.14, F6).

G8-B1 (dangling reference) deserves a note, because it is the check that a reviewer would not perform manually and a tool performs trivially. A requirement citing RULE-001, where RULE-001 does not exist, is a traceability hole that reads as traceability. The reference implementation's own documentation uses this exact case as its worked example: a requirement that "rests on a rule nobody ever defined" is treated by the completion gate as a hole rather than ignored.

7.10 Gate 9 – completion

Precondition: G8 passed. Output: the final validation report.

Rule	Severity	Condition
G9-B1	**BLOCKER**	Any BLOCKER outstanding from Gates 1-8.
G9-B2	**BLOCKER**	Any CRITICAL unresolved source conflict.
G9-B3	**BLOCKER**	Any artifact in the registry is unparseable.
G9-B4	**BLOCKER**	Traceability query Q-Why fails for any implemented artifact. (Some implemented code has no reachable evidence.)
G9-B5	**BLOCKER**	Traceability query Q-Blast is non-deterministic. (Impact analysis is not reproducible.)
G9-W1	**WARNING**	Any DRAFT artifact referenced by an implemented task.
G9-W2	**WARNING**	Any UNKNOWN or ASSUMED artifact in a shipped path.
G9-W3	**WARNING**	No simplification audit performed.
G9-N1	NOTE	Undocumented assumptions in the shipped path.

9B5 is unusual and is included because impact-analysis non-determinism is a common and rarely diagnosed defect: when graph traversal order or conflict resolution depends on filesystem or map iteration order, two engineers asking "what does this change affect?" get different answers, and the traceability guarantee is void even though the graph is complete.

7.11 Gate summary

```
Gate  Name           Primary blocker
----  -------------  ---------------------------------------------------
G1    context        Project purpose and project type undeclared
G2    research       Evidence without resolvable, dated, rated source
G3    domain         Concept without behaviour, rule, or requirement
G4    requirements   Requirement without derivation path or testable form
G5    opportunity    Specification advanced without opportunity record
G6    specification  Specification without requirement, or prescribing how
G7    plan           Component without justifying requirement
G8    implementation  Laundered epistemic state; dangling reference
G9    completion     Q-Why or Q-Blast fails for the shipped system
```

Appendix B gives the full precondition tables.

8 REJECTING THE MINIMUM VIABLE PRODUCT

8.1 Position

The framework does not adopt the Minimum Viable Product as a scope-governing device. This is a deliberate, argued decision, recorded here in the format the framework itself requires, and it is one of the points on which the framework most distinguishes itself from conventional product process.

8.2 What MVP does well

The MVP addresses a genuine and important defect in plan-driven sequential delivery: scope is committed before the feedback that would invalidate the commitment is available. Its contribution is to make the learning-per-unit-effort ratio the governing quantity, and to bound the cost of being wrong. This is correct, and the spiral model [13] anticipated it.

8.3 Why its preconditions fail here

MVP assumes, without exception, four preconditions.

```
P1. The problem is sufficiently known to define a deliverable at all. MVP
    scopes *a solution*; it presupposes that the solution shape is known well
    enough to cut.
P2. The unknown is market response, not domain semantics. MVP's learning
    variable is user reaction to a working artefact.
P3. The feedback loop is fast relative to the cost of a wrong assumption.
P4. Value can be delivered in a thin slice and accumulated.
```

Each fails in the target setting, and the failures are not marginal.

P1 fails directly. Section 2 establishes that the dominant failure mode is semantic fabrication: the solution shape is precisely what is not known. An MVP cut from an unvalidated domain model does not risk wasted effort; it *fixes* the fabrication as a baseline against which all subsequent work is measured. This is worse than no MVP, because it converts an unknown into an apparently settled premise.

P2 fails. In the G2 setting – reconstruct a domain without access to experts – the unknown is regulatory obligation, business rule, actor decision right, and jurisdictional convention. None of these yield to user feedback, because users are not the authority on them and, importantly, users frequently do not know that they are being violated. A user testing a compliant invoicing system will report that it works.

P3 fails. Domain validation latency is dominated by source discovery, expert availability, and organisational decision, not by build time. In an AI-assisted regime build time approaches zero, so the ratio inverts: the expensive step is now the one MVP does not shorten. Optimising the quantity MVP optimises – learning per unit of engineering effort – becomes degenerate, because engineering effort is no longer the binding constraint.

P4 fails when the thin slice is not independently valuable. Many domain obligations are irreducibly collective: a ledger entry is not useful without the double-entry relation; a stock movement is not useful without the reservation that constrains it. Slicing these produces artefacts that are small and useless, which is the worst of both worlds.

8.4 What replaces it

Three mechanisms replace MVP, at different granularities.

8.4.1 Evidence-bounded capability classification

Every candidate capability is classified by evidentiary basis, not by appeal:

```
CORE          Required by a validated rule or a regulatory constraint.
              Omitting it produces a defective or non-compliant system.
IMPORTANT     Supported by evidence, addresses a validated need, not
              required for correctness.
OPTIONAL      Plausible; evidence is weak; requires a decision.
FUTURE        Justified in principle; not justified now.
OUT_OF_SCOPE  Explicitly excluded, with the reason recorded.
EXPERIMENTAL  Exists to answer a question, not to deliver value. Marked
              non-production. (Section 8.4.3.)
```

The classification is a function of the evidence state, not of delivery convenience. A capability cannot be classified CORE without a resolvable derivation path to evidence. This is the essential difference from MVP scoping: MVP asks "what is the smallest thing we can ship?", this asks "what does the evidence oblige us to build, and what is the evidence status of everything else?".

8.4.2 The evidence-complete increment

The unit of delivery is a vertical slice that is complete in *evidence*, not thin in *feature count*:

```
  one business rule
+ its epistemic state and evidence path
+ its specification, with error and boundary scenarios
+ its acceptance criteria
+ its verification mechanism
+ its impact record against existing artifacts
```

An increment is not complete because it compiles or because its tests pass. It is complete when the chain rule → requirement → specification → acceptance criterion → test is closed and each link is declared. This is *larger* than an MVP slice in the feature dimension, because it carries the whole chain, and *smaller* in the risk dimension, because the unverified surface is bounded by one rule rather than by an entire assumed product shape.

The trade is deliberate. MVP buys speed with unresolved semantics; the evidence-complete increment buys semantic closure at reduced parallelism. In a regime where code generation is cheap, this is the correct direction of exchange and the framework is explicit that it is a trade, not a free improvement.

8.4.3 MVP retained where it is actually valid: the experiment

MVP's underlying technique – ship a thin artefact to a limited audience to obtain feedback quickly – is excellent, and is retained inside Gate 5 under a different name and a different obligation.

```
The Wizard-of-Oz MVP becomes EXP-001, an experiment artifact.
Its purpose is to answer a stated question, not to deliver value.
Its scope is bounded and its non-production status is mandatory.
Its result is VALIDATED / PARTIALLY_VALIDATED / REJECTED / INCONCLUSIVE /
NEEDS_MORE_EVIDENCE / PIVOT.
An INCONCLUSIVE result may not be recorded as VALIDATED (Gate 5, G5-W3).
A validated capability still requires the full evidence chain before it
becomes a requirement; the experiment is evidence *about* the concept, not a
substitute for the requirement's derivation.
```

This is the correct place for MVP thinking in the framework, and the reason for relocation is precise: an experiment is a knowledge-production device, and the framework permits knowledge-production devices only inside the stage whose purpose is knowledge production, with their outputs subject to that stage's falsification rules.

8.5 Consequences accepted

Rejecting MVP has costs, and the framework records them rather than rationalising them.

```
C-1  Slower time to first deployable output, because the first increment
     carries a full evidence chain.
C-2  Lower parallelism in early increments, because each touches the
     knowledge substrate.
C-3  Requires access to sources, or to a human able to make business
     decisions. Where neither exists, the framework will correctly report a
     high volume of UNKNOWN, and the project will appear blocked. It will be
     blocked: that is the accurate report.
C-4  Requires the discipline to record "no material opportunity" rather than
     proceeding, which is socially harder than proceeding.
```

C-3 is the binding constraint and deserves emphasis. In the G2 setting, with no experts and weak sources, the framework's most likely output is a large uncertainty registry and a small set of CORE requirements. This is a worse project plan and a better project. The framework's contribution is to make that distinction legible rather than to conceal it.

8.6 What would falsify this position

Consistent with the framework's own epistemics, the position is stated with its refutation condition. The MVP rejection is wrong if a controlled comparison (S2, Section 13) shows that MVP-scoped systems achieve equal or better correctness against validated requirements, at lower cost, in a setting where domain knowledge is *not* the binding constraint. We expect this to be true in low-domain-entropy settings – pure UI work, well-specified algorithm implementation, greenfield tools with no external obligations – and expect it to be false in the setting the framework targets. The framework should therefore be applied selectively, by gate outcome, and its applicability is an OPEN empirical question (RQ5).

9 THE TDD / BDD / SDD PROCESS

9.1 The error this section corrects

The TDD, BDD and SDD literatures are usually presented as a pipeline:

```
write specification (SDD) -> write feature files (BDD) -> write unit
tests (TDD)
```

This is wrong in two ways, and both errors are costly in the agentic setting.

The first error is sequence. The three are not successive phases; they are concentric loops of different radii that run *concurrently*, each with its own artifact type, its own unit of verification, and its own definition of done. A team that treats them as phases performs them in that order and then maintains three sets of artifacts that describe the same behaviour at three levels of redundancy, drifting apart.

The second error is scope. Presented as a pipeline, SDD appears only at the front, as documentation produced before implementation. In reality SDD is the outermost loop and it is the only loop that continues after the code is written, because the specification remains the authority against which drift is detected. A pipeline presentation cannot express this, and so cannot express drift detection.

9.2 The three loops, defined

Loop	Radius	Primary artifact	Unit of verification
SDD	system / feature	SPEC + AC (Markdown)	acceptance criterion, reviewed by a human
BDD	component	.feature + step defs	scenario, executed against a real boundary
TDD	unit	test fn + domain object	function contract, no I/O

Loop	Test process	Definition of done
SDD	none (declarative)	AC is measurable; main, alternative, error and boundary scenarios exist; links to REQ resolve
BDD	acceptance suite, full stack	scenario green against the assembled application boundary, not a mock of it
TDD	unit suite, no I/O	red -> green -> refactor, mutation-resistant

Three properties of this table are load-bearing.

First, the three definitions of done are *different* and none substitutes for another. "The unit tests pass" is not "the scenario passes" is not "the acceptance criterion is reviewed". Three teams reporting "tests pass" may be reporting three different things, and the ambiguity is invisible because the phrase is shared.

Second, SDD has no test process. Its verification is human review. This is a weakness and it is the reason the BDD loop exists: BDD is the mechanism that converts SDD's reviewed prose into something executable, which is why BDD's value is *proportional to how much specification there is*, not to how much code.

Third, the unit of verification differs in kind, not only in size. An acceptance criterion is a claim about observable behaviour at a boundary; a function contract is a claim about a transformation in isolation. Substituting one for the other produces either untestable specifications or tautological tests. Section 9.8 gives the detectable signatures.

9.3 The outer loop: SDD

Cycle time: one specification increment, typically hours to days. Artifacts: SPEC, AC, SCN. No code, no test.

1. Select the AC set for this increment from a validated REQ.
2. Write the main scenario in observable terms: actor, boundary, state before, state after.
3. Write the alternative, error, and boundary scenarios. This step is not optional and is the single most-skipped part of specification practice in the field; Gate 6 (G6-B3) exists because of it.
4. Make each AC measurable. "The system SHALL be fast" fails; "the system SHALL return the allocation within 200 ms at p95 under 500 concurrent reservations" passes.
5. Close Gate 6.
6. Hand the AC set to the BDD loop. The handoff is mechanical and is performed by the provisioner (Section 10.7), not by hand.

The SDD loop does not close when the code works. It closes when the AC set is reviewed and unambiguous, whether or not any code exists. Spec-first mode [2] terminates here.

9.4 The middle loop: BDD

Cycle time: one scenario, minutes. Artifacts: .feature, step definitions, World/harness.

1. The feature file is *derived* from the AC set, tagged with the AC id (Section 10.7). It is not hand-written, because a hand-written feature file has no guarantee of corresponding to any AC.

2. Step definitions bind each step to an application port. The generated World object exists to make this structurally enforced: step definitions receive a harness, never a database handle.
3. The scenario runs. On first run every step is a deliberate failure carrying the full provenance chain in its message. This is the loop's red step, and it is the correct initial state.
4. Implement until green, at whatever level the behaviour requires.
5. Refactor the implementation; the scenario must not move.

The constraint in step 2 is what earns BDD its cost. If step definitions may reach directly into persistence, a green scenario proves nothing about the application's behaviour; it proves that the database contains a row. The indirection through a port is the whole justification for the layer.

Corollary, stated because it is frequently ignored: BDD is the *wrong* level for a library with no application boundary. There is no boundary to observe, so a scenario can only reach past it. Section 10.13 makes the provisioner refuse in that case rather than generate hollow scenarios.

9.5 The inner loop: TDD

Cycle time: minutes. Artifacts: unit test, domain object, test double.

Standard red-green-refactor [11], with two constraints specific to this framework.

Constraint 1 – the unit test traces to a RULE, not to an AC. This is the detail most often got wrong and it corrupts the traceability model when mistaken. The two levels have genuinely different provenances:

```
RULE-004  --verifies-->  TDD unit test
AC-014-01 --verifies-->  BDD scenario
SPEC-014  --contains-->  both
```

The unit test's justification is the business rule it enforces. The scenario's justification is the acceptance criterion that describes the behaviour at the boundary. A unit test that cites AC-014-01 is asserting that a level-2 specification is also a level-1 contract, which is false in general and tempting always.

Constraint 2 – the unit test runs with no I/O. Persistence, network and clock are injected. A unit test that touches a database is not a unit test; it is a slow integration test that has lost the ability to localise failure.

9.6 Handoff contracts

From	Field	To	Consumed as
SDD	AC id, scenarios, rules	BDD	generated feature + tags
SDD	AC id, boundary, errors	TDD	not consumed directly
BDD	observed green/red per AC	SDD	drift signal, Gate 8
BDD	application port signature	TDD	the seam to implement
TDD	rule coverage report	BDD	confidence in the domain
Plan	interface contract	BDD	contract suite
Plan	interface contract	TDD	type/interface tests

The row "BDD → SDD: observed green/red per AC" is the feedback edge that makes SDD a loop rather than a phase, and it is the mechanism by which specification-to-code drift is detected per increment rather than per audit.

9.7 Rigour selection

BDD, TDD, contract testing, property-based testing, model-based specification and formal verification are not stylistic alternatives. They are mechanisms with different applicability conditions, different cost profiles, and different classes of defect they can detect. A framework that mandates one globally prescribes taste and calls it engineering.

OpenDomain therefore makes verification-rigour *selection* an engineering decision, made per requirement against stated criteria, and recorded as an artifact.

Requirement character	Primary mechanism	Secondary
Business behaviour, acceptance	BDD / Gherkin	Contract test
Unit-level contract	TDD	Property-based test
REST interface	OpenAPI + schema test	Pact
GraphQL interface	GraphQL SDL	Pact
gRPC interface	Protocol Buffers	Pact
Event interface	AsyncAPI	Consumer-driven test
Service compatibility	Pact / Specmatic	Integration
State-heavy lifecycle	State machine / model	Model-based
Mathematical invariant	Property-based	Formal (if tractable)
Performance envelope	Benchmark + load test	Synthetic data
Security property	Adversarial test	Static analysis
Uncertain / contested rule	Scenario, not test	Recorded as UNKNOWN

9.8 Three misuses and their detectable signatures

M1 BDD used for unit testing. Step definitions assert on internal structures; scenarios name data structures rather than domain states; a single class is named in most scenarios.
Signature: a Given clause naming a data structure ("a row in stock") rather than a domain state ("an item with 100 on hand"). Checkable.

M2 TDD used as acceptance. Test names mirror requirement text; the suite is green while the behaviour at the boundary is absent or wrong.
Signature: unit suite green and zero BDD scenarios exist for the AC set. Gate 8 (G8-B4) reports every AC as uncovered, which is the correct and sufficient detection.

M3 SDD used as documentation. Specification written or edited after the implementation, revised to match what the code does.
Signature: a specification file and an implementation file changed in the same commit. This is mechanically detectable from version-control history and is a BLOCKER, added as G6-B6. The check is crude and deliberately so: it will occasionally fire on a legitimate correction, and the cost of that false positive is one review comment, against a real cost of silent documentation-after-the-fact.

9.9 The contested-rule row

The final row of the selection table is the one most often omitted. When a behaviour depends on a rule whose epistemic status is ASSUMED or CONFLICTING, the correct verification artefact is not a test but a *scenario with the uncertainty attached*. Writing a test asserts the rule. Writing a scenario that names the rule as contested records that the behaviour is a decision.

The mechanical consequence is a prohibition the framework does enforce: a test may not be the justification of a requirement. Justification flows downward from evidence; verification flows upward from code. A requirement whose only support is a test asserting it is a closed loop, and Gate 8's G8-B3 exists partly to make such loops visible.

9.10 Executable specifications

Where practical, specifications are executable: Gherkin features linked to step definitions, OpenAPI documents validated against implementations, property-based tests for invariants true by construction rather than by example. The framework prefers executable specification because it makes specification and test the same artifact, removing one drift class.

The framework's caution is the one expressed in [2]: generated code is a proposal requiring verification, not a derivation with guaranteed correctness. Executable specifications bind specification to verification. They do not validate the specification's correctness against the domain, which is a Gate 4 concern and which no amount of green scenarios can discharge.

10 AUTOMATIC VERIFICATION TOOLCHAIN PROVISIONING

10.1 The provisioning problem

Given the process in Section 9, the tool must answer five questions on every project: which test runner, which BDD framework, which contract framework, how step definitions are wired, and how tests are linked to acceptance criteria.

Left to an agent, each of the five is answered wrongly in a characteristic way.

```
Q1  Wrong runner. The agent installs a framework that does not match the
    project's build system, or a second runner beside the existing one, so
    that the project's tests and the framework's tests cannot see each other.
Q2  Global install. The agent installs a tool globally. The toolchain stops
    being reproducible and the project stops being self-describing -- the
    exact failure the artifact-substrate stance exists to prevent.
Q3  Hand-written step definitions. The agent writes step definitions that
    reach past the application boundary into the database, producing green
    scenarios that prove nothing.
Q4  Unlinked tests. The agent writes tests that cite no AC. This is the
    catastrophic one: it breaks the chain on which the entire framework's
    value depends. Tests that cannot be traced to criteria are exactly as
    untraceable as requirements that cannot be traced to evidence, and the
    framework would then be silent about the most important class of defect
    it exists to detect.
Q5  Drifted feature files. Feature files are authored once and never
    re-derived, so the specification and the executable specification
    describe different systems.
```

None of these are agent failures in the sense of carelessness. They are consequences of leaving a structural decision unstructured.

10.2 Six provisioning principles

```
PP1  Detect, do not ask. The stack is mechanically detectable. Asking a user
     which test runner they use is a question the tool can answer, and
     Section 5.8 reserves human attention for decisions only a human can
     make.

PP2  Never global. Project-local manifests only, exact version pins. A
     globally installed test framework makes the project's verification
     unreproducible from the repository, which contradicts the framework's
     position that the artifacts must survive the tool.

PP3  Idempotent and declared. Provisioning state lives in od.toml.
     Re-running converges to a fixed point and produces no diff. A
     provisioning command that is not idempotent is not safe to put in a
     setup script, and will not be run.
```

```
PP4  Derive structure, never author behaviour. The provisioner generates
     files, tags, empty step-definition stubs that fail, type stubs, and
     contract skeletons. It never generates a passing assertion. This is a
     safety property, not a limitation: an agent asked to "make the tests
     pass" must be structurally unable to comply by writing assertions, and
     must instead read the specification and implement the behaviour. Without
     this property, "add BDD and make it green" is a request the agent can
     satisfy without ever reading the specification, which defeats the
     framework's entire purpose.

PP5  Traceability is derived, therefore drift-free. Scenario tags are
     generated from the AC id, and re-synchronisation re-derives them from
     the specification. There is no path by which a scenario silently
     detaches from its criterion except by editing a generated tag, which is
     a visible diff in a generated file.

PP6  Provisioning is subject to the same anti-overengineering discipline as
     any other construct. Section 10.13 defines the refusal conditions, and
     refusal produces a recorded decision rather than silence.
```

10.3 P0 – stack detection

Detection reads only the repository. No network, no prompting.

Detect	From
Language(s) and versions	Cargo.toml, pyproject.toml, package.json, pom.xml, build.gradle, go.mod, *.csproj, Gemfile
Package manager and lockfile	uv.lock, poetry.lock, package-lock.json, Cargo.lock, go.sum, *.lock
Existing unit runner	dev-dependencies, test directory layout
Existing BDD framework	manifest, features/ tree
Test layout convention	existing test path shape
CI system	.github/workflows, .gitlab-ci.yml, Jenkinsfile
Application boundary	presence of an entry point, a port/module with an inbound interface, or a server manifest

The CI detection is not incidental. A provisioned suite that is not registered in CI does not run, and a verification mechanism that does not run is a document. Provisioning therefore writes the CI registration as part of the same operation, and records having done so.

The detection result is written to od.toml as a `Stack` record and is an artifact with epistemic = INFERRED and confidence MEDIUM, because a misdetected stack is an inference error. The provisioner prints its detection and reasoning in the phase report, and provides an explicit override (`od provision -stack=...`) so that a human can correct an inference without editing generated files. A misdetection that the framework cannot see and correct is worse than no detection.

10.4 P1 – deterministic framework selection

Selection is by ordered rule, first match wins. The agent is not asked to choose and is not permitted to invent a framework; a choice outside the table is an error, not a preference.

```
R1  A BDD framework already present in the manifest -> adopt it, install
    nothing. (Brownfield preservation; overriding an existing team's
    deliberate choice is the most common unforced harm in this subsystem.)
R2  Else, the language has an established Cucumber/Gherkin port -> use it.
R3  Else, or if the project has no business-actor-facing behaviour -> do not
    provision BDD. Record the decision and the reason (Section 10.13).
R4  Else -> shim mode: a thin Gherkin parser and driver over the project's
    native runner, for stacks with no port.
```

```
Stack                Unit (TDD)                BDD                        Contract
↪ Property-based
-------------------  ------------------------  -------------------------  --------------
↪ ---------------
Rust + cargo         built-in #[test]          cucumber-rs                pact-reference
↪ proptest
Python + pytest      pytest                     pytest-bdd / behave        pact-python
↪ hypothesis
JS/TS + npm          vitest (or existing jest)  @cucumber/cucumber-js;      @pact-foundation
↪ fast-check
                                                 playwright-bdd (UI)        /pact
Java + Maven/Gradle  JUnit 5                   cucumber-jvm               pact-jvm        jqwik
C# + .NET            xUnit                     SpecFlow                    pact-net
↪ FsCheck
Go                   std testing               godog                       pact-go         rapid
Ruby                 RSpec / Minitest          cucumber                    pact-ruby       rspec
PHP                  PHPUnit                   behat (Gherkin)             pact-php        --
```

10.5 The observation that makes this cheap

Every BDD framework in the table above is a port of Cucumber, and every one runs on its language's *native* test runner. Two consequences, both of which matter for the framework's cost case.

First, step definitions and unit tests share one process, one mocking library, one coverage tool, and one debugger. The BDD/TDD boundary is therefore not an operational wall: the same fixtures, the same test doubles, the same CI job. The indirection costs indirection, not a second toolchain.

Second, Gherkin is the portable part of the artifact. Generated feature files are portable across the entire column; only step definitions are language-bound. A project that changes language keeps its executable specifications. This is a small instance of a general principle – the specification outlives the tooling – and it is not obvious before it is noticed.

10.6 P2 – installation

1. Resolve the project's own package manager. Never a second one.
2. Add the pinned dependency through that package manager, so that the lock file is updated by the tool that owns it.
3. Write the decision to od.toml AND emit a `ProvisioningDecision` artifact (kind DEC) recording: stack detected, frameworks chosen, rules R1-R4 applied, versions pinned, and the epistemic status of the detection.
4. Converge: a second run produces no diff.
5. On failure – no network, unavailable registry, version conflict – provisioning FAILS LOUDLY, writes nothing partial, and records the unavailability as a limitation. It must not emit a manifest that cannot resolve, and it must not silently proceed without verification.

Step 3 deserves emphasis. An installation decision *is* a decision, and the framework records decisions (Section 5.9 forbids hiding architectural decisions in code). A provisioner that silently picks a BDD framework is making an undocumented architectural decision on the project's behalf – the exact anti-pattern the framework criticises elsewhere. Every automatic choice becomes a reviewable artifact.

10.7 P3 – wiring generation

```
Layout derived from bounded-context scope:


features/
  inventory/
```

```
    reservation.feature        <- derived from SPEC-014, tagged with AC ids
  procurement/
tests/
  bdd/
    inventory/
      reservation_steps.rs     <- stubs, red by construction
  contract/
tests/
  unit/                        <- TDD, annotated or inferred binding
```

Scope in the path is not cosmetic. A feature file's directory *is* its scope, so the generator refuses to write a scenario whose specification scope differs from its directory. A scenario about procurement cannot be filed under inventory, and the attempt is an error rather than a judgement call.

```
Step-definition stubs are generated empty and failing, carrying their own
provenance in the failure message:
```

```rust
#[given("an item with {on_hand:d} on hand and {reserved:d} already reserved")]
fn given_stock(on_hand: i64, reserved: i64, _w: &mut World)
    -> Result<(), StepError>
{
    todo!("AC-INVENTORY-014-01 <- SPEC-014 <- RULE-004 <- EVD-001. \
           Derive from the specification; do not infer from this string.")
}
```

Three properties. The body is `todo!` (or `NotImplementedError`, or `pytest.fail`) – a deliberate failure, never a plausible-looking assertion. The failure message carries the full derivation chain, so that whoever runs the suite first is told where the truth lives rather than what the string says. And the step receives a `World` carrying a harness, never a database handle, which is what makes the green scenario meaningful (Section 9.4).

10.8 P4 – traceability binding, and what it buys

```
Every generated scenario carries a tag per criterion and a provenance
comment:
```

```
@AC-INVENTORY-014-01 @SPEC-INVENTORY-014
# derives-from: RULE-004, REQ-INVENTORY-014, EVD-001
Scenario: Reject an over-reservation that exceeds available quantity
  Given an item with 100 on hand and 30 already reserved
  And a request to reserve 80
  When the reservation is requested
  Then the reservation is rejected
  And the allocatable quantity remains 70
```

At audit time the provisioner parses tags, builds TEST- artifacts keyed by AC id, executes the suite, records the per-criterion result, and writes the edges `AC -verifies-> TEST` and `TEST -verifies-> implementation`.

Four consequences, which are the actual payoff:

```
(a) G8-B4 becomes a measurement rather than a declaration. "An acceptance
    criterion has no test" is detected by the absence of a tagged scenario;
    and a tagged scenario that is *red* is a stronger finding still, because
    it means the specification and the code disagree right now.

(b) Regeneration cannot drift. Change the AC, run `od provision --sync`, the
    tag follows. Detachment requires editing a generated file.
```

```
(c) The framework can answer a question no static artifact set can: not "is
    this requirement implemented" but "is this requirement implemented *and
    does its acceptance criterion currently hold in the running system*". That
    is a dynamic property, and it is the closest the framework comes to the
    empirical validation of Section 5.12's G3.


(d) Test coverage becomes a query over the graph rather than a report
    someone regenerates.
```

10.9 Verification depth – the necessary caveat

The binding above is only as strong as the suite's ability to run. A project whose scenarios require external services may not be executable at audit time. The framework therefore records a `verification-depth` on every TEST- artifact and in every audit report:

```
executed    the scenario was run and its result observed now
discovered  the scenario exists and is bound; it was not run
declared    a link is asserted in an artifact; nothing was observed
```

Audit output always states which depth produced it. A project reporting `discovered` coverage has not demonstrated that anything works, and the framework must not present it as though it had. This is the false-confidence error of Section 4.2, caught at the point where it is easiest to catch – in the reporting layer, where coverage numbers are most often produced and least often qualified.

10.10 P5 – unit-level binding, and binding provenance

The unit level needs its own binding mechanism, and it needs an honest account of how good that mechanism is.

Method	Mechanism	Binding	Confidence
ANNOTATED	`// od:RULE-004` comment in the test	explicit	HIGH
ATTRIBUTED	language-native marker attribute	explicit	HIGH
INFERRED	naming convention matched to glossary	inferred	LOW

ANNOTATED via comment is the default because it requires no build-system change, works in every language, and survives refactoring of the test harness. The convention fallback exists because a large existing test suite will not be annotated, and a traceability graph with no unit-level edges is much less useful than one with honestly-labelled inferred edges.

This forces a small but important extension to the artifact model. The `verifies` relation carries a `binding` attribute – GENERATED, ANNOTATED, or INFERRED – and the framework never presents the three as equivalent. A link derived by matching a function name against a glossary term is a hypothesis about intent, and marking it as though it were a generated tag would be epistemic laundering performed by the framework itself.

10.11 P6 – contract testing

Where a specification carries a CTR artifact with a machine-readable source (OpenAPI, AsyncAPI, GraphQL SDL, Protocol Buffers), the provisioner additionally generates:

- a provider verification test skeleton derived from the document: every operation, every required field, every documented status code, the authorisation requirement;
- a consumer contract derived from the consumer's actual usage, if the consumer is in the workspace;
- CI registration of the pact verification step;

- a breaking-change check, which is the highest-value and cheapest item here:

```
git show HEAD~1:openapi.yaml | od contract diff

which fails on any breaking change. Removing a response field or widening a
constraint is a breaking change, and in agent-assisted development this
is a class of regression that no unit test catches because the tests are
regenerated with the code.
```

10.12 Brownfield: extract, do not install

The most common real case, and the one where restraint matters most.

1. Install nothing. The project already has a toolchain and a team's accumulated choices; changing it is a cost with no benefit.
2. Walk the existing test tree. Extract from tags, comments, naming, and test metadata.
3. Bind by ANNOTATED where possible; fall back to INFERRED and mark confidence LOW.
4. Report the binding rate as the project's traceability baseline.
5. Gate nothing on it in the first pass.

Step 5 is a deliberate exception to the framework's own gate discipline, and the exception is justified. A project with a 20% inferred binding rate has just been told something about itself that was previously unknown, and failing its build for its history is not the useful response. The useful response is to make the 20% visible, agree it, and raise it incrementally. The framework's job here is to *measure what was invisible*, not to enforce a standard retroactively.

This number is a deliverable. A project that learns its traceability baseline is 20% and that 60% of its tests cannot be connected to any acceptance criterion has received genuine information that no amount of green CI would have given it.

10.13 The provisioning gate

Provisioning is refused, with the decision and reason recorded, when:

- fewer than the configured minimum behavioural acceptance criteria exist (default 3). BDD for one scenario is indirection with no payoff.
- no use case with a named actor exists (Section 9.4's corollary).
- the change is marked EXPERIMENTAL and non-production.
- the project is a library with no application boundary.
- the project has declared the BDD-off profile.

Refusal must emit a decision record naming the alternative mechanism – a naming-convention binding, or scenario artifacts without execution. A refusal that produces silence is indistinguishable from an oversight.

The BDD-off profile deserves separate statement, because a framework that mandates a style commits the error it criticises in Section 4.7. The profile is a first-class, declared mode in which the team accepts the indirection cost and prefers acceptance-criterion identifiers carried in test names. The framework supports it completely, and reports its coverage honestly at the correspondingly lower verification depth. Declaring the profile is a decision record; using it is not a violation.

10.14 Limits and failure modes

Stated plainly, because this subsystem is where the framework is most likely to disappoint.

```
F1  Provisioning produces a red suite. Correct by design and by PP4, but a
    team that runs the provisioner, sees N failures, and has not read the
    specification will conclude the tool is broken. Mitigation: the
    provisioner reports "N scenarios generated, all red by design" as a
    first-class result, never as a project failure.
```

```
F2  Cucumber-family step registries are process-global. Two bounded contexts
    defining "an item with X" collide at runtime, and the failure appears far
    from its cause. Mitigation: generated step definitions are namespaced by
    scope and the generator rejects ambiguous definitions. This is the most
    frequently hit limitation in practice.

F3  BDD carries real indirection cost. Teams that find it painful will drop
    it, and history suggests many do. Mitigation: the BDD-off profile. We
    regard this as a genuine risk to adoption of the whole framework, not
    only of this subsystem.

F4  Version coupling. The BDD framework, the native runner, and the language
    version move together. Pinning controls this but does not remove it, and
    upgrades require re-verification -- a maintenance cost the framework
    imposes and must justify (Section 17.3).

F5  THE MOST IMPORTANT LIMIT. Auto-provisioning improves the *chain*, not the
    *content*. If the acceptance criterion is vague, the generated tag is
    precise and the generated scenario faithfully encodes the vagueness. The
    provisioner can make an unspecified criterion consistent, tagged, and
    permanently green. It cannot make it specified. Provisioning must never
    be allowed to create the impression that a verified chain validates its
    own inputs, because a chain of tagged, well-formed, green scenarios over
    a vague criterion is the most reassuring artefact the framework can
    produce and it means nothing. Gates 4 and 6 remain the only defences, and
    they operate on content, not on tags.

F6  Supply-chain exposure. Automatic installation of test frameworks is an
    attack surface, and a compromised test dependency executes with the
    project's privileges. Mitigation: exact version pins, recorded in the
    decision artifact, no global installation, and an installation diff that
    is reviewable in the same commit as everything else.

F7  The provisioner makes decisions. Every automatic choice is a decision and
    is recorded as one (Section 5.9). Without this the subsystem would be
    the largest single source of undocumented architectural decisions in the
    framework, which is intolerable.
```

10.15 The adapted lifecycle, end to end

One evidence-complete increment, on a Rust project, driven by one business rule. This is Sections 8.4.2 and 9 and 10 operating together.

```
# once, on the repository
od init
od provision
  # P0 detects Rust + cargo + no BDD framework
  # P1 rule R2 -> cucumber-rs; unit = built-in; contract = pact-reference
  # P2 pins versions, emits ProvisioningDecision
  # P3 writes features/inventory/, tests/bdd/inventory/, CI registration
  # P4 binds existing SPEC/AC set; reports: 0 criteria, nothing to generate
  # "provisioned: <cucumber-rs @ pinned>, <pact-reference @ pinned> (local)"
  # version strings are resolved at provisioning time and recorded in the
  # lockfile and in the ProvisioningDecision; they are deliberately not
  # hard-coded here, because a paper that pins them would go stale silently
```

```
# --- the SDD loop, closing ---
od research stock-reservation
od requirement new "distinguish on-hand from allocatable" --from EVD-001
od spec new SPEC-INVENTORY-014 --from REQ-INVENTORY-014 --rigour spec-anchored
od review-spec SPEC-INVENTORY-014
```

```
od gate 6
  # AC measurable? error + boundary scenarios present? not co-committed
  # with implementation? -> passes

# --- the BDD loop, seeded by the provisioner, red by design ---
od provision --sync SPEC-INVENTORY-014
  # derives features/inventory/reservation.feature from the AC set
  # tags @AC-INVENTORY-014-01 @AC-INVENTORY-014-02 @AC-INVENTORY-014-03
  # writes steps whose every body is todo!() carrying the derivation chain
cargo test --features bdd
  # 3 scenarios, 3 steps each, all red. Correct initial state.

# --- the TDD loop, innermost, minute cycle ---
# author unit tests bound to RULE-004 (not to AC-014)
// od:RULE-004
fn test_reservation_cannot_exceed_allocatable() { ... }
# implement the domain rule until green; refactor; the unit test does not move

# --- the BDD loop goes green against the real boundary ---
cargo test --features bdd
  # green. If red, the specification and the code disagree: that is the
  # drift signal, available per increment, not per audit.

# --- the chain closes, mechanically ---
od gate 7 && od gate 8
  # G8-B2 laundering check
  # G8-B4 every AC has a scenario
  # G8-B7 provisioning drift: tag set == AC set
od audit --verification executed
od trace --why REQ-INVENTORY-014
    # REQ-014 <- EVD-001 <- SRC-001 (authority: HIGH)
od impact AC-INVENTORY-014-01
```

Two properties of this sequence are worth naming, because they are what the provisioning subsystem buys.

The BDD scenario is red *before* any unit work begins, and stays red until the boundary behaviour genuinely exists. That is the TDD red step applied at the scenario level, and it is what makes the three loops one process rather than three: no loop can be satisfied by a different loop's work.

And the AC-to-test link was never written by anyone. It was generated from the specification id. The chain REQ ← EVD ← SRC closes at Gate 9 not because a disciplined author maintained a matrix, but because the artifacts were derived from one another.

11 REFERENCE IMPLEMENTATION ARCHITECTURE

11.1 Purpose and stance

The reference implementation (`opendomain`) exists to make the gates executable and the queries answerable. It is deliberately not an IDE, not a requirements management system, and not a code generator. Its entire function is to make the framework's prohibitions checkable and its traceability queries mechanical.

Design stance: the artifact store is plain Markdown with fenced structured blocks, held in the repository. This is a strong decision with three justifications. It is version-controllable and therefore diffable. It is agent-editable without a tool. And it survives the tool: the knowledge substrate does not become inaccessible if the framework is abandoned, which is the principal failure mode of tool-coupled process.

11.2 Component decomposition

```
opendomain-core
  vocab       Closed value sets: knowledge types, artifact statuses,
              confidence, authority, scope, stage
  ids         Identifier parsing, validation, scoped identifiers,
              acceptance-criterion inheritance
  artifact    Artifact and document model; the 13-relation vocabulary
  parse       Markdown + TOML block extraction into artifacts
  workspace   Directory discovery, registry indexing, document assembly
  graph       Traceability graph, Q-Why traversal, Q-Blast impact analysis,
              deterministic ordering
  gates       The nine gates and their findings
  scaffold    Workspace initialisation and template materialisation
  provision   Stack detection, framework selection, idempotent local install,
              wiring generation, @AC tag extraction, binding provenance,
              the refusal conditions of Section 10.13
  report      Terminal and JSON rendering
  config      Workspace configuration
  error       Error type


opendomain-cli  (`od`)
  init, gate, audit, trace, impact, index, report, spec, validate, doctor
  provision [--check] [--sync] [--profile <name>]
```

The provision module is the only component permitted to execute a package manager, and it is the only component whose effects are not confined to the workspace's own artifacts. Its purity exemption is explicit and narrow: the provisioner writes to the dependency manifest and the lockfile, and those two files are the workspace state it may touch. It may not modify application source. G8-B7 exists because this is the one place where the framework's automatic behaviour changes the project's build, and that change must be visible in review.

11.3 The purity constraint

The central implementation constraint, stated in the core crate's own documentation, is that every operation is a pure function of the workspace on disk. `od audit` and `od gate` never mutate. Running them twice gives the same answer.

This constraint is not stylistic. A checker that mutates cannot be trusted to judge, because its judgement alters what it judges, and re-running it yields a different verdict. Reproducibility is what allows a gate to be used as a blocking condition in an automated pipeline and what allows two engineers to disagree productively about a finding rather than about a version.

11.4 Determinism

Because Q-Blast is a blocking condition at Gate 9 (G9-B5), traversal order must be specified, not inherited. All iteration over sets and maps in impact analysis is performed in a canonical order derived from identifier kind and identifier value. This is the only place in the implementation where an unspecified detail is a correctness defect rather than a style issue, and it is called out here because it is easy to introduce and hard to diagnose.

11.5 Extension points and their limits

The framework is notation-agnostic. Where a project requires SysML or a domain-specific modelling language, a projection exports artifacts into the target notation; artifacts remain the source of truth. Compatibility with the DSML approach of [3] is therefore complementary rather than competing: the knowledge and its provenance live in artifacts, and the notation is a view.

11.6 Implementation status

Stated precisely, per Section 5.12.

The reference implementation is a working PROTOTYPE of the core: artifact model, parser, workspace index, graph, gates, scaffold, and report modules are present and internally documented, and the core crate's documentation carries an executable example of gate evaluation.

The verification subsystem of Section 9 and the provisioner of Section 10 are NOT implemented. This is a deliberate consequence of the paper-first ordering and is recorded here as a limitation of the current artifact rather than as a plan. Specifically absent: any feature-file or contract generation, any `@AC` tag extraction, any binding-provenance record, the G6-B6, G8-B5, G8-B6 and G8-B7 checks, the `tests` relation's use as a binding edge (Section 5.4), and the `provision` module itself. Every provisioning example in Section 10 is a specification of intended output, not a transcript of observed output.

The CLI crate (`opendomain-cli`, `od`) is implemented, with all subcommands described in the framework (init, status, trace, impact, graph, gate, audit, link, show, advance, propose, apply, archive, reference) and JSON output support where applicable.

The workspace builds cleanly. The core crate's unit tests pass (91 passed, 2 doctests passed) with no warnings, and `cargo fmt --all -- --check` is clean, under Rust 1.98.1 at edition 2021. Every worked example shipped in the templates is loadable by the parser: each identifier prefix resolves to a registered artifact kind, and each `status` value is one the parser accepts. A template whose examples the parser rejects would be a trap, because uncommenting one invalid heading makes every `od` command fail for the whole workspace rather than for the offending file. The two defects recorded in earlier drafts of this section -- a manifest edition unsupported by the installed toolchain, and public re-exports naming symbols absent from the vocabulary module -- are fixed.

The existence of this section is the framework working as intended.

11.7 Traceability of this paper to the implementation

Claim in this paper	Implemented artifact
Closed epistemic lattice (5.2)	vocab::KnowledgeType
7 epistemic values (5.2)	vocab, 7 variants
13 artifact statuses (6.2)	vocab::ArtifactStatus::ALL
13 relations (5.4)	artifact::Relation::ALL
37 identifier kinds (6.1)	ids::IdKind
9 gates (7)	gates::Gate::ALL
Gate 8 dangling-reference (G8-B1)	gate-8/dangling-reference (1)
Purity of gate evaluation (11.3)	gates::evaluate
Deterministic impact analysis (11.4)	graph::ImpactReport
Structured block parsing (11.1)	parse::parse
Minimal-artifact scaffolding (5.11)	templates/ (40 files)
CLI (init, status, trace, impact, graph, gate, audit, link, show, advance, propose, apply, archive, reference)	opendomain-cli

(1) KNOWN DISCREPANCY, RECORDED RATHER THAN PAPERED OVER. This paper numbers the gates 1-9 (Section 7): Gate 8 is "implementation", Gate 9 is "completion". The implementation numbers them 0-8, so its `gate-8` is "completion", the paper's Gate 9, not the paper's Gate 8. The dangling reference rule therefore lives in the completion gate in code, whereas Section 7 places it in the implementation gate. The rule ID is reproduced above exactly as it appears in the source, because a traceability table that silently renames an identifier to match its author's intent is the defect this paper argues against. Reconciling the two -- either by moving the rule to the implementation gate or by restating Section 7 -- is a design decision and is not made here.

Claim in this paper	Status
Three nested loops (9.2)	NOT IMPLEMENTED -- design only
Executable specification generation (9.10)	NOT IMPLEMENTED -- design only
Rigour-by-requirement-type table (9.7)	NOT IMPLEMENTED -- design only
Stack detection (10.3)	NOT IMPLEMENTED -- design only
Framework selection rule (10.4)	NOT IMPLEMENTED -- design only
Idempotent local installation (10.6)	NOT IMPLEMENTED -- design only
Wiring generation (10.7)	NOT IMPLEMENTED -- design only
Binding provenance (10.8, 10.10)	NOT IMPLEMENTED -- design only
Provisioning refusal gate (10.13)	NOT IMPLEMENTED -- design only
G6-B6, G8-B5, G8-B6, G8-B7	NOT IMPLEMENTED -- design only

The second table is included because a traceability table that lists only implemented claims is the same defect this paper argues against: it presents a designed framework as a working one. Every row above the second table is a claim a reader can execute against the repository today. Every row in the second table is a claim the reader cannot, and the framework's own standard requires that the difference be visible rather than inferred from silence.

12 EPISTEMIC STATUS OF THIS PAPER

The framework requires that uncertainty be visible. Applied to itself:

Claim	Status	Confidence
Semantic fabrication is a real failure mode of AI coding assistants	INFERRED	MEDIUM
Structure presence masks semantic absence	INFERRED	MEDIUM
MVP preconditions fail in the target setting	INFERRED	MEDIUM
The nine gates detect the defects claimed	INFERRED	LOW
The nine gates have acceptable false-positive rate	UNKNOWN	UNKNOWN
OpenDomain reduces unfounded requirements	UNKNOWN	UNKNOWN
OpenDomain reduces rework or defect rate	UNKNOWN	UNKNOWN
OpenDomain is more efficient than MVP-scoped development	UNKNOWN	UNKNOWN
Domain knowledge in RE is established as significant, and its formalisation, acquisition and maintenance are recognised challenges	EXPLICIT [1]	HIGH
Specification authority should be tiered by consequence and duration	EXPLICIT [2]	HIGH
Domain knowledge is not carried by modelling notation alone	INFERRED [1][3]	MEDIUM
Existing MBSE approaches without standards anchoring are of limited practical applicability	EXPLICIT [3]	HIGH

No empirical result is claimed. Sections 13 and 14 specify how the open claims would be closed. A reader who requires empirical support should treat Sections 5 through 11 as a design proposal and Section 12 as the honest summary of its status.

13 EVALUATION PROTOCOL

Four studies. S1 and S2 are primary; S3 and S4 are supporting.

13.1 S1 – Longitudinal case study (addresses RQ1, RQ2, RQ6)

Design: single-case, embedded, longitudinal [24]. Unit of analysis: the change set. One greenfield project and one brownfield project, 3-6 months, minimum 60 change sets.

Data: every change set classified on four measures.

```
M1 Provenance completeness. Proportion of introduced requirements and domain
   rules with a resolvable evidence path. Ground truth established by
   independent review by a domain-experienced practitioner, blind to the
   framework's classification, to avoid circularity.
M2 Epistemic accuracy. Agreement between the framework's declared epistemic
   state and the independent practitioner's judgement. Measured as a
   confusion matrix over the seven values, reported in full.
M3 Unfounded-requirement rate. Requirements whose derivation path terminates
   at a competitor or at agent prior, as determined by review.
M4 Overhead. Wall-clock and token cost attributable to the framework, measured
   as the difference between framework-assisted and unassisted work on matched
   change sets.
```

Analysis: within-case time series with change set as unit; intervention points marked at each framework version change, to separate learning effects from tool effects. This matters: the first weeks of any new process show inflated cost and inflated compliance, and attributing either to the framework is a standard error.

Threats: single-case generalisability; the reviewing practitioner's own bias; the strong likelihood that novelty effects inflate early compliance. Mitigated but not eliminated; see Section 14.

13.2 S2 – Controlled comparison against MVP scoping (addresses RQ5)

Design: within-subject crossover, alternating blocks. Teams or individuals implement matched change sets under three regimes:

```
R-A  Evidence-complete increments (Section 8.4.2)
R-B  MVP scoping (thin slices, learning-per-effort optimisation)
R-C  Unstructured AI-assisted development (baseline)
```

Outcome measures, in order of importance:

```
O1 Correctness against independently validated domain requirements. Not
   self-reported test pass rate. The requirements are validated by the
   practitioner before implementation begins and are not shown to
   implementers.
O2 Regret. Changes required post-deployment, weighted by the domain
   criticality of the violated rule.
O3 Cost. Total engineering time and token cost.
O4 Traceability query latency and correctness for Q-Why and Q-Blast.
O5 Change failure rate.
```

The primary endpoint is O1, and specifically O1 measured against *externally validated* requirements. This is the design's central commitment: the evaluation must not accept the system's own specification as the measure of the system's correctness, which is the false-confidence error identified in [2] and reproduced here as a measurement-design constraint rather than merely a warning.

Prediction: R-A will show higher O1 and lower O2, and worse O3 in the first increment, converging over time. R-C will show the highest O2. A result in which R-B dominates on O1 falsifies Section 8 and is reported as such.

13.3 S3 – Epistemic-state audit (addresses RQ1, RQ3)

Design: mechanical. For every project adopting the framework, extract the epistemic state transition history of every artifact and classify each transition.

Measures: distribution of transition types; rate of ASSUMED/INFERRED → VALIDATED transitions accompanied by a new evidence item (the laundering rate, the framework's key metric); time-to-close for artifacts in UNKNOWN and CONFLICTING; and the fraction of shipped-path artifacts that were never promoted past their initial epistemic state.

The last measure is the framework's most uncomfortable and most informative: a high fraction of never-promoted artifacts in a shipped path means the process produced accurate records of its own ignorance and shipped anyway, which is a legitimate outcome under uncertainty and a failure under compliance. Reporting it as a single number would be misleading; it is reported conditional on the project's compliance obligations.

13.4 S4 – Gate utility (addresses RQ3, RQ6)

Design: adversarial. Construct a corpus of workspaces seeded with known defects, one per gate condition, plus controls. Measure per gate: true-positive rate, false-positive rate, and the time for a competent engineer to reach a correct disposition on a BLOCKER finding.

The false-positive measurement is the important one. A gate set that blocks on defects that are not defects is a gate set that will be disabled, and a disabled gate set is worse than no gate set because it creates the appearance of rigour. A target of under 5% false-positive rate per gate condition is stated in advance; exceeding it is treated as a design defect requiring revision, not as noise.

13.5 Instrumentation

Every gate execution emits a machine-readable report. The reports form the telemetry for S3 and S4 with no additional instrumentation, which is a deliberate property of the purity constraint in Section 11.3: a non-mutating checker produces observational data as a by-product of doing its job.

14 THREATS TO VALIDITY

14.1 Threats to construct validity

- M2 (epistemic accuracy) uses a human practitioner's judgement as ground truth, and that judgement is itself fallible and may share biases with the framework's categories. Mitigated by blind independent review and full confusion-matrix reporting, but not eliminated.
- M4 (overhead) attributes cost to the framework when part of the cost is learning the tool. Mitigated by the crossover design in S2 and by reporting overhead as a time series rather than a mean.
- The gate set may be complete for the failure modes this paper identifies and incomplete for others. S4 tests only the seeded corpus; it cannot establish coverage of unknown defect classes.

14.2 Threats to internal validity

- Novelty and training effects are the dominant confound in any process intervention. A framework that is being used for the first time will show inflated compliance and inflated cost. Mitigated by blocked crossover and time-series analysis; never fully eliminated in a field setting.
- Self-reported behavioural measures are subject to social desirability, since the framework makes adherence publicly visible in the repository. Mitigated by relying preferentially on mechanical measures (M1, M3, and all of S3/S4) over self-report.
- Selection effects: projects that adopt the framework are likely to differ systematically from those that do not. Not addressed by the protocol; stated as a limitation on external validity.

14.3 Threats to external validity

- The domain-entropy assumption. The framework's advantage is predicted to increase with domain entropy – the degree to which correct behaviour depends on knowledge external to the codebase. For low-entropy work (UI, algorithms with given specifications) it may impose cost for no benefit. This is predicted, not measured, and is the most likely boundary of the framework's applicability.
- Single-region, single-sector evidence. If S1 is conducted in one regulatory and organisational context, the framework's regional-context discipline (Section 5.8) is untested.
- The framework has not been evaluated with a domain expert in the loop, which is the setting in which its authors expect the best cost-benefit ratio. The evaluation setting is therefore the harder case, which strengthens but does not replace the missing measurement.

14.4 Threats to conclusion validity

- Multiple endpoints across S1-S4 with no pre-registered primary endpoint invites selective reporting. Mitigation: O1 in S2 is designated primary in advance and the remaining measures are declared secondary here.
- The framework's author is also its evaluator. Full mitigation requires independent replication, which is stated as future work (Section 18) rather than claimed.

15 DISCUSSION

15.1 What the framework changes about the agent's role

The framework does not make the agent more capable of generating code. It makes the agent's *assertions* checkable.

Concretely: the agent's fluent output is admitted into the knowledge substrate only through a path that requires it to declare what it is doing – extracting from a source, inferring, assuming, or not knowing. An agent that says "I don't know" can proceed, because UNKNOWN is a valid state that blocks a specific gate and is therefore actionable. An agent that cannot say "I don't know" can only proceed by asserting, and its assertions are now checkable against declared evidence.

This reframes the alignment question. The concern is usually expressed as "the model will hallucinate". The framework's contribution is more specific: it makes the *epistemic status of every claim in the delivered system* an inspectable property, so the question becomes not "did the model hallucinate" but "which claims are unsupported, and do we know which ones those are".

15.2 The framework as a research instrument

A secondary consequence is that adopting OpenDomain produces data about AI-assisted engineering that is otherwise unavailable. The artifact substrate records, per claim, the epistemic status at which it entered, the evidence cited, and the transitions it underwent. This yields a dataset on the distribution of epistemic states in generated requirements, which Section 12 notes is currently UNKNOWN and which would be a contribution independent of the framework's own effectiveness.

15.3 The cost objection, taken seriously

The most credible objection to this framework is cost. Every evidence record, uncertainty entry, decision record, and gate check is overhead, and overhead competes with delivery. Three responses, stated without rhetorical embellishment.

First, the cost is front-loaded and the benefit is back-loaded, which is the worst possible shape for adoption under deadline pressure and the best possible shape for correctness over a system's

life. In the AI-assisted regime, the asymmetry widens: generation is cheap, so the relative cost of the knowledge substrate falls even as its absolute value rises.

Second, the minimal-artifact principle bounds the cost directly. The framework does not require forty documents; it requires that any document be justified. The reference implementation's forty-five template files are a menu.

Third, and most honestly: the framework's cost is highest exactly where it is most needed. In the low-entropy setting it is overhead. This is a real cost, it is predicted rather than measured, and Section 8.6 states the condition under which the framework's central position is wrong.

15.4 On rejecting MVP

The MVP rejection is the framework's most contestable position and deserves to be stated as an argument a reader can reject.

The argument is not that MVP is wrong. It is that MVP presupposes a solved domain, and the framework's target setting is defined by an unsolved domain. In a setting where the problem is sufficiently known and the unknown is user response, MVP is the correct instrument and this framework is overhead. In a setting where the unknown is what the rules are, MVP's first cut is a commitment to a fabrication, and the commitment is rewarded by every subsequent process step. The framework declines to make that commitment and pays for the refusal in traceability work.

Whether that refusal pays is RQ5, and it is open.

15.5 The three-stage separation

The framework preserves a separation the source protocols insisted on and that this paper formalises:

```
DISCOVER    What is true of the world?
CHALLENGE   What could be better, and what would falsify that belief?
SPECIFY     What exactly must the software do?
IMPLEMENT   Realise the specification.
VALIDATE    Demonstrate alignment, and audit the alignment itself.
```

The failure mode this guards against is the collapse of DISCOVER into SPECIFY, which in an agentic setting is the default, because an agent asked "what should this system do?" will answer, confidently, without having determined what is true. The framework makes that answer inadmissible as a first act.

16 LIMITATIONS

```
L1  The framework is untested. Every performance and effectiveness claim is
    OPEN. Section 12 is the definitive statement.


L2  It requires sources. A project with no access to documentation, code,
    databases, or knowledgeable people will receive a large UNKNOWN report and
    a small set of CORE requirements. The framework will not manufacture
    knowledge, and no amount of process will substitute for it.


L3  It requires organisational permission to record uncertainty. Teams under
    delivery pressure have strong incentives to mark artifacts VALIDATED. The
    framework detects status edits without evidence changes, but detection
    after the fact is not prevention, and a team that does not want the
    record will find ways around the gate.


L4  Overhead in the low-entropy case is unjustified and predicted, not
    measured.
```

```
L5  The gate set is a design proposal. Coverage and false-positive rates are
    unknown. S4 exists to measure them and has not been run.

L6  Notation-agnosticism, while principled, costs specificity. A project
    requiring a formal DSML must build the projection themselves, and nothing
    in the framework validates that projection.

L7  The brownfield path is the least developed. Reverse specification with
    behaviour classification is specified but has the least supporting
    material in the source protocols and the least implementation support.

L8  No multi-agent coordination protocol is specified. The framework states
    that agents may work in parallel on non-conflicting tasks and that shared
    contracts must precede parallel work, but the contract-negotiation problem
    across agents is unaddressed.

L9  The framework is authored by its proponent. This is stated in Section 14.4
    and is not mitigated.

L10 The verification subsystem of Section 9 and the provisioner of Section 10
    are specified but not implemented. No claim is made that the three loops
    improve defect rates, that the rigour table produces better test
    targeting, or that automatic provisioning reduces setup cost. The design
    argument for all three is an argument, and Section 13 specifies the study
    that would test it.

L11 The framework selection matrix in Section 10.4 is a judgement, not a
    measurement. It encodes what is mainstream and maintainable per language
    as of the version date, and it will decay as ecosystems move. The matrix
    is deliberately data, external to the gate logic, so that a wrong entry
    is correctable without changing the framework; the cost of that choice is
    that matrix quality is now a maintenance burden the framework does not
    discharge.

L12 Automatic provisioning creates a supply-chain surface the framework does
    not fully control. Pinning and local installation reduce but do not
    eliminate it, and a compromised test dependency executes with project
    privileges. This is recorded as F6 in Section 10.14 and gated by G8-B7,
    which detects the undocumented case only.
```

17 ETHICS AND SUSTAINABILITY

17.1 Epistemic honesty as an engineering duty

The framework's central ethical claim is narrow and defensible: in domains governed by regulation, safety, or fiduciary obligation, a system whose assertions are unfounded is a harm mechanism, and the profession's obligation extends to making the unfoundedness visible rather than to producing well-documented systems whose documentation does not indicate its own reliability.

This is a claim about legibility, not about correctness. The framework does not make software more correct in the first instance; it makes incorrectness identifiable. In a regulated setting this is the difference between a defect and an unknowable defect.

17.2 The risk of the framework being misused

- Ceremonial adoption. Producing the artifacts without performing the thinking. This is the failure mode of every process framework and is worsened by automation, which makes the artifacts cheap. Mitigation: the epistemic accuracy measure M2 in S1, which detects precisely this, since artifacts that are wrong about their own epistemic state are worse than absent artifacts.

- Weaponised uncertainty. Marking a required capability OUT_OF_SCOPE or UNKNOWN to avoid building it. Mitigation: the classification of Section 8.4.1 makes CORE require a derivation path, so deferral is visible as deferral.
- Manufactured authority. Citing a source that does not support the claim. Mitigation: G2-B2 requires the source to be rated authoritative *for the claim*, which requires a per-claim judgement that is difficult to fake at scale and becomes visible in review.

17.3 Sustainability of the knowledge substrate

The artifacts are plain text, in the repository, under the project's own licence. The knowledge outlives the tool. This is a deliberate design choice with an environmental and professional dimension: a process that requires a proprietary tool to maintain its own records will not be maintained, and the records will be lost, which is a worse outcome than never having created them.

The framework also has a resource dimension worth stating. The most expensive resource it consumes is expert attention, and the discipline of Section 5.8 is directed at that expense: ask the human only for decisions that require a human. A framework that routes vocabulary questions to domain experts has consumed the scarcest input in the system and produced nothing.

18 FUTURE WORK

```
F1  Independent replication of S2 by a group without framework
    involvement. This is the only sufficient answer to Section 14.4.
F2  Gate coverage analysis: a systematic mapping of failure modes in
    AI-assisted engineering to gate conditions, following the method of [1]
    applied to the framework's own checks.
F3  Automated epistemic-accuracy estimation: methods for estimating the
    correctness of a declared epistemic state without the reviewing
    practitioner, to reduce the cost of the mechanism the framework depends
    on most.
F4  Multi-agent coordination protocol: contract negotiation, ownership
    arbitration, and conflict detection when several agents write to the same
    knowledge substrate.
F5  Brownfield support: automate behaviour classification to the extent that
    static and dynamic analysis can support it, and be explicit where it
    cannot.
F6  Domain-entropy estimation: a measurable predictor of when the framework
    pays and when it does not. This would convert Section 14.3's boundary
    condition from a prediction into a decision rule.
F7  Formalisation: express the gate conditions in a form suitable for
    mechanical proof or model checking, so that the framework's guarantees are
    provable rather than tested. The initial target is the epistemic-lattice
    transition system, for which G8-B2 is currently the only enforcement.
F8  Integration with domain-specific modelling languages: an export from the
    artifact graph into a DSML such as that of [3], preserving the separation
    between knowledge provenance and notation.
F9  Longitudinal outcome data: defect escape rates, rework, and change
    failure rates over multi-year periods, which is the only timescale on
    which the framework's central bet -- that epistemic closure compounds --
    can be evaluated.
F10 Implementation of the verification subsystem: the rigour table, the
    generated executable specifications, the binding-provenance record, and
    the provisioner, in that order, each gated on the previous. G6-B6 should
    not be enabled before the artifacts it checks can be generated, or the
    gate will fail on every requirement for a reason unrelated to the
    requirement.
F11 Rigour-effectiveness study: an ablation comparing projects that bind
    tests to acceptance criteria against projects that do not, holding the
    requirement set fixed. Until this runs, Section 9's central claim -- that
    the rigour column changes test targeting rather than merely documenting
```

```
    it -- is INFERRED.
F12 Provisioning-failure corpus: a maintained record of detection and
    selection failures across real repositories, which is the only way to
    revise the matrix of Section 10.4 on evidence rather than on opinion, and
    to measure the false-refusal rate of the gate in Section 10.13.
F13 Binding-inference evaluation: a measurement of how often name-based
    inference of a test-to-criterion binding is correct, which determines
    whether INFERRED bindings may ever satisfy G8-B6 without human
    confirmation.
```

19 CONCLUSION

AI coding assistants have made the production of code cheap. They have not made the determination of what to produce any easier, and by lowering the cost of the first they have raised the expected cost of the second: a wrong system is now cheap to build and expensive to discover.

This paper has argued that the binding problem is not that models fabricate, which they do, but that fabrication is indistinguishable from competence in the output, because the output carries no record of how any of it is known. The proposed remedy is not a better prompt or a longer process document. It is a representational change: make epistemic state part of the data, attach provenance to every claim, and make the resulting structure checkable by a pure function over a version-controlled workspace.

OpenDomain makes that change in four commitments. The epistemic lattice, which permits ignorance to be recorded and forbids it from being implicit. The evidence gate, which forbids an external statement from becoming a requirement without passing through finding, interpretation, and candidate need. The traceability spine, which makes "why does this exist?" and "what breaks if this changes?" graph traversals rather than recollections. And the nine gates, which convert the tractable prohibitions of a written method into checks that block.

It adds two commitments that are less conventional. An opportunity stage that makes innovation a falsifiable, kill-criteria-governed engineering process rather than a feature-list exercise. And a formal rejection of the Minimum Viable Product as a scope-governing device, on the grounds that its preconditions suppose a solved domain, which is precisely what the target setting lacks – with the honest note that the empirical claim is open, that the argument is refutable, and that in low-domain-entropy settings the framework is likely to be overhead.

None of this has been shown to work. The reference implementation does not currently build. The failure analysis is inferred from observation rather than measurement, and the effectiveness claims are OPEN. We state this because the framework's central claim is that unfounded assertion is the primary hazard, and a paper making unfounded effectiveness claims would fail its own first test. What is offered here is a falsifiable design, a precise specification, an evaluation protocol, and an honest inventory of what remains unknown.

References

[1] M. Araujo, J. Araujo, R. Oliveira, L. Romao, M. Kalinowski, "Domain Knowledge in Requirements Engineering: A Systematic Mapping Study," in Proc. 51st Euromicro Conf. Series on Software Engineering and Advanced Applications (SEAA), 2025. arXiv:2506.20754.

[2] D. B. Piskala, "Spec-Driven Development: From Code to Contract in the Age of AI Coding Assistants," arXiv:2602.00180, submitted to AIWare 2026.

[3] L. Beers, H. Nabizada, M. Weigand, A. Chahine, F. Gehlhoff, A. Fay, "A Domain-Specific Modeling Language for Production Systems in Early Engineering Phases," Systems, vol. 14, no. 2, art. 150, 2026. doi:10.3390/systems14020150

[4] E. Evans, Domain-Driven Design: Tackling Complexity in the Heart of Software. Boston, MA: Addison-Wesley, 2003.

[5] M. Fowler, “Microservices,” martinfowler.com/articles/microservices.html, 2014.

[6] E. Evans, “Microservices – a definition of this new architectural term,” martinfowler.com/bliki/Microservices.html, 2014.

[7] M. Fowler, “Anemic Domain Model,” martinfowler.com/bliki/AnemicDomainModel.html, 2003.

[8] I. Jackson, Domain-Specific Languages: Design, Implementation and Analysis. Boston, MA: Addison-Wesley, 2005.

[9] A. Cockburn, Writing Effective Use Cases. Boston, MA: Addison-Wesley, 2003.

[10] G. North, “Specification by Example,” Better Software, vol. 11, no. 3, 2006.

[11] K. Beck, Test-Driven Development: By Example. Boston, MA: Addison-Wesley, 2003.

[12] G. Meszaros, xUnit Test Patterns: Refactoring Test Code. Boston, MA: Addison-Wesley, 2007.

[13] B. W. Boehm, “A Spiral Model of Software Development and Enhancement,” IEEE Computer, vol. 21, no. 5, pp. 61-72, 1988.

[14] B. W. Boehm and R. Turner, Balancing Agility and Discipline: Scaling Lean and Agile to Large Systems. Boston, MA: Addison-Wesley, 2004.

[15] P. Zave, “Classification of Research Efforts in Requirements Engineering,” ACM Computing Surveys, vol. 29, no. 4, pp. 315-321, 1997.

[16] B. Ramesh and F. T. Jarke, “Toward Reference Models for Requirements,” IEEE Computer, vol. 34, no. 6, pp. 44-51, 2001.

[17] P. Naur, “Programming as Theory Building,” Communications of the ACM, vol. 28, no. 5, pp. 26-31, 1985.

[18] H. A. Simon, The Sciences of the Artificial, 3rd ed. Cambridge, MA: MIT Press, 1996.

[19] Object Management Group, “Unified Modeling Language (UML) Specification,” omg.org.

[20] Object Management Group, “Systems Modeling Language (SysML) Specification,” omg.org.

[21] INCOSE, Systems Engineering Handbook, 5th ed. Hoboken, NJ: Wiley, 2023.

[22] B. Kitchenham and S. Charters, “Guidelines for Performing Systematic Literature Reviews in Software Engineering,” 2007.

[23] C. Wohlin, Guidelines for Writing and Analyzing Empirical Studies in Software Engineering. Lund, Sweden: Lund University, 2000.

[24] R. K. Yin, Case Study Research and Applications: Design and Methods, 6th ed. Thousand Oaks, CA: Sage, 2018.

[25] Cucumber, “Gherkin Reference,” cucumber.io/docs/gherkin/.

[26] OpenAPI Initiative, “OpenAPI Specification,” spec.openapis.org.

[27] Pact Foundation, “Contract Testing Documentation,” pact.io.

[28] GitHub, “Spec Kit,” github.com/github/spec-kit.

[29] M. Kleppmann, Designing Data-Intensive Applications. Sebastopol, CA: O’Reilly, 2017.

[30] S. He, Y. Feng, M. Galan, “Requirements, Architecture, and Design: The Role of Technical and Non-functional Requirements in Microservices,” Journal of Systems and Software, 2018.

A Appendix A – ARTIFACT FIELD SCHEMAS

A.1 Common fields (all kinds)

```
id          Required. Stable, typed, scoped. See Section 6.1.
kind        Required. One of 37 identifier kinds.
```

```
title       Required. One line.
status      Required. One of 12 values.
epistemic   Required. One of 7 values. No default.
confidence  Optional. HIGH | MEDIUM | LOW. Confidence in the inference.
authority   Optional. Per-claim authority rating, not per-source.
owner       Required for rules, concepts, decisions, requirements.
scope       Required when the project has more than one bounded context.
created     Required.
updated     Required.
version     Required once modified after approval.
relations   Zero or more of 13 relation values, each naming an existing id.
```

A.2 Kind-specific requirements

```
SRC      url (required), access-date (required), source-type (required),
         authority (required), coverage (required)
EVD      claim (required), source (required, must resolve to SRC),
         exact-context (required), interpretation (required),
         affected-concepts, affected-requirements
TERM     definition (required), synonyms, disambiguation
RULE     statement (required), type (required), scope (required),
         affected-process, affected-entities, validation-method
REQ      statement (required), rationale (required), actor (required),
         priority, acceptance-criteria, derived-from (required, at least one
         of evidence, rule, or explicit ASSUMED marking)
NFR      quality-attribute (required), measure (required), threshold
         (required). A non-functional requirement without a threshold is
         invalid.
UC       primary-actor (required), goal (required), trigger,
         main-scenario (required), alternative-scenarios,
         exception-scenarios, postconditions
SPEC     intent, preconditions, trigger, main-scenario (required),
         alternative-scenarios, error-scenarios, business-rules,
         state-changes, security-constraints, performance-constraints,
         data-constraints, acceptance-criteria, contracts
AC       criterion (required), must be measurable, given/when/then
CTR      protocol, direction, request, response, errors, auth, versioning,
         idempotency, failure-behaviour
TASK     objective (required), source-specs (required), dependencies
         (required), acceptance-criteria (required), tests-required
ADR      context, problem, options, evidence, decision, consequences,
         rejected-alternatives, status
HYP      statement (required), target-population, metric (required),
         threshold (required), experiment, failure-condition (required)
EXP      question, hypothesis, method, inputs, metric, threshold, cost,
         duration, result, decision
UNC      question (required), why-unknown, evidence-available,
         possible-interpretations, impact, validation-required, status
XCONF    topic (required), source-a, source-b, difference,
         possible-explanation, project-decision, status
```

A.3 What is deliberately absent

No field for priority-as-delivery-order, no field for story points, no field for "confidence in the estimate", no field for "percent complete". These belong to a different concern and admitting them into the knowledge substrate would let delivery pressure write into epistemic records.

B Appendix B – GATE PRECONDITION TABLES

B.1 Preconditions

```
G1  (none)                          -> project-context.md
```

```
G2  G1 passed                    -> sources, evidence, glossary,
                                    uncertainties, conflicts, log
G3  G2 passed                    -> actors, stakeholders, entities,
                                    events, processes, rules, domain-map,
                                    ownership
G4  G3 passed                    -> requirements, nfr, constraints,
                                    baseline
G5  G4 passed                    -> opportunities, assumptions,
                                    contradictions, concepts,
                                    hypotheses, experiments, cdr
G6  G5 passed                    -> specs, acceptance-criteria, contracts
G7  G6 passed                    -> architecture, components, adr, plan,
                                    tasks
G8  G7 passed                    -> code, tests, drift records
G9  G8 passed                    -> audit report, final validation
```

B.2 A note on gate ordering

G5 (opportunity) follows G4 (requirements) rather than preceding it. The alternative order – opportunity before requirements – was considered and rejected: opportunity analysis conducted against an unvalidated domain model produces concepts that are attractive and unfounded, and the framework has no mechanism to detect that except the absence of the domain model itself.

The chosen order has a cost: projects that are genuinely exploratory must establish a requirement baseline before they are permitted to explore. The mitigation is that G4's classification explicitly includes OUT_OF_SCOPE, and a project may baseline "the current state" as its requirement set and proceed. This is a real cost, recorded rather than concealed.

C Appendix C – WORKED MICRO-EXAMPLE

C.1 The scenario

An assistant is asked: "add stock reservation to our inventory module so we stop overselling."

C.2 Gate 2 (research) – what the sources say

```
SRC-001  Warehouse management reference documentation.
         Authority: HIGH for generic WMS workflow.
         Status: active.
SRC-002  The organisation's own purchase procedure (internal).
         Authority: HIGH for this organisation's receiving rules.
SRC-003  A competitor's product page stating "reservation supported".
         Authority: MEDIUM for market practice; LOW for this
         organisation's requirements.

EVD-001  Claim: Warehouse systems commonly record stock movements and
         distinguish available from reserved quantities.
         Source: SRC-001.
         Exact context: "Reserved stock is not available for allocation."
         Interpretation: The distinction between available and reserved
         quantity is common in this class of system.
         Epistemic: EXPLICIT.  Confidence: HIGH.
         Affected concepts: StockItem, Reservation.

EVD-002  Claim: Reservation may be taken before or after payment depending on
         the organisation.
         Source: SRC-001, SRC-002.
         Exact context: the two sources do not agree on ordering.
         Interpretation: The lifecycle position of reservation relative to
         payment is CONTESTED and cannot be resolved from public sources.
         Epistemic: CONFLICTING.  Confidence: MEDIUM.
         Affected concepts: Reservation, Order, Payment.
```

```
XCONF-001  Topic: reservation position relative to payment.
           Source A: SRC-001 (reservation precedes payment).
           Source B: SRC-002 (payment precedes reservation).
           Difference: ordering of two events in the order lifecycle.
           Possible explanation: organisational financing policy.
           Project decision: PENDING.
           Status: UNRESOLVED.
```

Note that EVD-002 and XCONF-001 are the framework doing its central work. The agent's likely output without them is a confident lifecycle with a fixed ordering, chosen silently.

C.3 Gate 4 (requirements) – the evidence gate in action

```
Failed promotion (recorded to demonstrate the gate):
  Source: SRC-003. Finding: a competitor supports reservation.
  -> REQ: "The system SHALL support stock reservation."
  REJECTED: no Candidate Need record. The competitor's capability is
  evidence about the market, not about this organisation's problem.


Successful derivation:
  EVD-001 -> Candidate Need: users need to know what is actually allocatable,
  as distinct from what is physically present.
  -> REQ-014: The system SHALL distinguish, for each item, the quantity
     physically on hand from the quantity allocatable, and SHALL expose both.
  Epistemic: EXPLICIT. Derived from: EVD-001, RULE-004.


Blocked requirement:
  -> REQ-015 (candidate): The system SHALL hold a reservation from order
     confirmation until shipment.
  Status: BLOCKED by XCONF-001. Epistemic: UNKNOWN.
  Resolution required: business decision, or resolution of the conflict.


RULE-004: A reservation may not be created for a quantity exceeding the
allocatable quantity.
Epistemic: INFERRED from EVD-001. Consequence: Gate 4 passes this rule
because it is explicitly marked INFERRED rather than asserted as EXPLICIT.
A later reconciliation can promote it; nothing has claimed it is confirmed.
```

C.4 Gate 6 (specification) – acceptance criteria

```
AC-014-01: Given an item with 100 on hand and 30 already reserved
            And a request to reserve 80
            When the reservation is requested
            Then the reservation is rejected
            And the allocatable quantity remains 70


AC-014-02: Given an item with 100 on hand and 0 reserved
            And a request to reserve 100
            When the reservation is requested
            Then allocatable quantity becomes 0
            And a subsequent request for any positive quantity is rejected


AC-014-03: Given a reservation against an item
            When the reservation is released
            Then allocatable quantity increases by the reserved quantity
            And the movement is recorded in the item's movement history


Spec-014 is a BDD/Gherkin specification. Its three criteria cover the normal
case, the boundary case at zero availability, and the release case, which is
the reversal path. Gate 6 checks that a behaviour with a failure mode has an
error or boundary scenario; AC-014-01 and AC-014-02 discharge that check.
```

C.5 Gate 8 (implementation) – the laundering check

```
Suppose an engineer, having shipped the feature, edits REQ-014 from
epistemic = INFERRED to epistemic = EXPLICIT without adding an EVD record
citing a source that states it. Gate 8 rule G8-B2 fires:


gate-8/epistemic-laundering
REQ-014 promoted INFERRED -> EXPLICIT with no intervening evidence


This is the check that the framework exists to provide. The engineer has not
written a false requirement; they have made a supported requirement look
confirmed. Without the check, the distinction is invisible, and the
distinction is the entire point.
```

D Appendix D – MAPPING TO THE FOUNDING OPERATIONAL PROTOCOLS

D.1 Domain-engineering protocol (two variants)

Protocol section	-> Framework element
Never jump from sources to code	Gate ordering; Section 5.1
Domain precedes software	G3 before G4 before G6
Evidence is not a requirement	Section 5.3; G4-B1
Competitor feature != requirement	G4-W1; Section 5.3 worked example
Never silently invent rules	Epistemic lattice; G2-B3
Source registry	SRC kind; G2-B2
Evidence registry	EVD kind; G2-B1
Glossary / conflict / uncertainty	TERM, XCONF, UNC kinds
Actor, stakeholder, process, rule	ACT, STK, PROC, RULE kinds
Cross-domain analysis, ownership	Section 5.6; G3-B3, G3-W3
Domain events	EVT kind; G3-N1
Requirement classification	REQ, NFR, CNST, BAS kinds
Requirement quality check	G4-B3, G4-B4, G4-W3
Requirement challenge	G4-W1, G4-W2
Use case engineering, UML	UC kind; G3-W1; G6
Domain model, class and state	ENT, EVT kinds; G3-B4
Architecture, anti-overengineering	G7-W1; Section 5.9
ADR	ADR kind; G7-B4
Implementation mapping, plan, tasks	PLAN, TASK kinds; G7-B1
Test engineering	Section 9; G6-B4, G8-B4
Traceability, impact analysis	Section 5.4; G9-B4, G9-B5
Global consistency audit	G9
Knowledge uncertainty	UNC kind
Decision log	DEC kind
Source conflicts	XCONF kind; G2-W1
Simplification audit	G9-W3
Final system audit	G9
Quality gates (6 gates)	Nine gates; Section 7

D.2 Spec-driven-engineering protocol

Protocol section	-> Framework element
Relationship to domain engineering	Section 4.8; Rule A
Source-of-truth hierarchy	G6-B2; G8-B2
Spec-first / anchored / as-source	Section 4.2; Section 5.12
Inspect project first	G1; G1-B3
Brownfield reverse specification	Section 5.10; G8-B2
Specification registry	SPEC kind; G6
Specification quality gate	G6-B1..B5
WHAT before HOW	G6-B2

Behavioural specification (Gherkin)	Section 9.4; Section 10.7
Specification types	CTR kind; G6-B5
Acceptance criteria	AC kind; G6-B4, G8-B4
Executable specifications	Section 9.10; Section 10.8
BDD / TDD	Section 9.2; Section 9.7
API-first, contract-first	CTR kind; G6-B5
Contract testing (Pact)	Section 10.4; Section 10.11
Model-based specification	Section 9.7
Technical planning	PLAN kind; G7
Architecture planning	G7-W1
Data and interface planning	G7-W3
Implementation decomposition	TASK kind; G7-B1, G7-B2
Dependency ordering	G7-B2
AI agent context isolation	Section 5.10
Incremental implementation	Section 8.4.2
Validation pyramid	Section 9.7
Bidirectional traceability	Section 5.4
Pre-requirements traceability	Section 5.5; G9-B4
Change management	G8-B1, G8-W2
Drift detection	G8-W3
Brownfield behaviour classification	Section 5.10
Self-specification	G6-B1
Specification review gate	G6
Plan review gate	G7
Implementation review gate	G8
Anti-overengineering	G7-B3, G7-W1
False confidence protection	Section 5.12; Section 12; S2 primary endpoint design
Tool selection matrix	Section 10.4
Definition of done	Section 8.4.2
Cross-module consistency	Section 5.6; G9
Traceability graph	Section 5.4
Minimal artifact principle	Section 5.11
Uncertainty management	Section 5.8; UNC kind

D.3 Opportunity / innovation-engineering protocol

Protocol section	-> Framework element
Research constrains invention	Section 4.8; Rule A
World A established / World B	Epistemic lattice; Section 5.2
Innovation pipeline (15 phases)	Gate 5
Current-state reconstruction	Gate 5 step 1
Pain / friction discovery	Gate 5 step 2
Assumption extraction	ASM kind; Gate 5 step 3
Contradiction analysis	CONTRAD kind; Gate 5 step 4
Opportunity space	OPP kind; Gate 5 step 5
Innovation operators (14)	Gate 5 step 6
Divergent exploration	Gate 5 step 6; G5-N1
Concept generation	CPT kind
Anti-innovation detection	G5-W2
Hypothesis formation	HYP kind; G5-B2, G5-W1
Experiment registry	EXP kind
Validation dimensions	G5-W3
Existing-solution comparison	Gate 5 steps 6-7
Innovation vs replication	G5-W2
Human-AI boundary	Section 5.2
Convergence gate	G5-B3
Concept decision record	CDR kind; G5-B3
Handoff to specification	G5-B1
Uncertainty management	Section 5.8
Falsification search	G5 step 8
Anti-overengineering gate	G7-W1
Innovation kill criteria	EXP decision values
Pivot mechanism	EXP decision PIVOT

```
MVP control / capability class.      Section 8.4.1
Boundary and handoff                 Section 5.11
```

D.4 Automatic verification toolchain provisioning

The provisioning subsystem of Section 10 is not one of the three founding protocols; it is the framework's own addition, and it is mapped separately so that its status is not confused with the mechanised material above. Every entry below is specified in this paper and NOT IMPLEMENTED (Section 11.6).

Provisioning element	-> Framework element
The specification outlives the tool	Section 5.4; AC kind
Project-local, pinned, idempotent	Section 10.6, PP2; G8-B7
Deterministic selection rule	Section 10.4, PP3
Existing framework preferred	Section 10.4, R1; Section 10.12
Generated stubs start red	Section 10.7, PP5
No hand-written test substitutes	Section 9.8, misuse signature 2
@AC tag as the binding key	Section 10.8; AC kind
tests relation as the binding edge	Section 5.4; Section 10.8
Binding provenance	Section 10.10; G8-B6
verification-depth field	Section 10.9; G8-B5
Rigour declaration must be executable	Section 9.7; G6-B6
Refusal to provision	Section 10.13; G8-W3
Supply-chain exposure recorded	Section 10.14, F6; G8-B7
Provisioner decisions recorded	Section 5.9; G8-B7
BDD-off profile	Section 10.13; G6-W3

The three-fold status of this table is the point. The three founding protocols are mapped to implemented gate conditions. This subsystem is mapped to designed conditions, most of which do not yet exist in code. Presenting the two in a single table without that distinction would be the same overclaim this appendix exists to prevent.

D.5 Net effect of the mapping

Of the 168 numbered protocol instructions in the three founding documents, the framework mechanises 61 as gate conditions. The remainder are classified as:

- not mechanisable (30): instructions requiring human judgement with no decidable predicate, e.g. "represent complex processes using diagrams", "avoid creating unnecessary classes";
- retained as review prompts (48): "consider", "evaluate", "question", phrasings whose value is attention-directing rather than checkable;
- superseded by the epistemic lattice (29): instructions against inventing domain knowledge, replaced by a single uniform mechanism.

The ratio is reported because it is a real limitation. A framework that claims to mechanise a written process method has usually overclaimed. The claim made here is narrower: that 61 of the tractable prohibitions are now checks, that the remainder are honestly classified, and that no instruction was silently promoted to a gate it does not support.

Two exclusions keep the count honest. The 61 counts gate conditions only, not the nine gates themselves and not the review prompts retained in the gate tables. It also excludes the six conditions introduced by Sections 9 and 10 (G6-B6, G6-W3, G8-B5, G8-B6, G8-B7, G8-W4), because none of them derives from the three founding documents; they are the framework's own additions and are mapped in D.4. Adding them to the 61 would improve the ratio and misstate where the conditions came from.